Spread

2026-03-27

SPREAD

Data setup

```r
library(urca)
library(tseries)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```r
library(quantmod)
```

```
## Loading required package: xts

## Loading required package: zoo

##
## Attaching package: 'zoo'

## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric

## Loading required package: TTR
```

```r
library(forecast)
library(rugarch)
```

```
## Loading required package: parallel
```

```r
library(FinTS)
```

```
##
## Attaching package: 'FinTS'

## The following object is masked from 'package:forecast':
##
##     Acf
```

```r
library(ggplot2)
library(dplyr)
```

```
##
## ######################### Warning from 'xts' package ##########################
## #                                                                             #
## # The dplyr lag() function breaks how base R's lag() function is supposed to  #
## # work, which breaks lag(my_xts). Calls to lag(my_xts) that you type or       #
## # source() into this session won't work correctly.                            #
## #                                                                             #
## # Use stats::lag() to make sure you're not using dplyr::lag(), or you can add #
## # conflictRules('dplyr', exclude = 'lag') to your .Rprofile to stop           #
## # dplyr from breaking base R's lag() function.                                #
## #                                                                             #
## # Code in packages is not affected. It's protected by R's namespace mechanism #
## # Set `options(xts.warn_dplyr_breaks_lag = FALSE)` to suppress this warning.  #
## #                                                                             #
## ###############################################################################

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:xts':
##
##     first, last

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
library(httpgd)
library(R.utils)
```

```
## Loading required package: R.oo

## Loading required package: R.methodsS3

## R.methodsS3 v1.8.2 (2022-06-13 22:00:14 UTC) successfully loaded. See ?R.methodsS3 for help.

## R.oo v1.27.1 (2025-05-02 21:00:05 UTC) successfully loaded. See ?R.oo for help.

##
## Attaching package: 'R.oo'

## The following object is masked from 'package:R.methodsS3':
##
##     throw
```

```
## The following objects are masked from 'package:methods':
##
##     getClasses, getMethods

## The following objects are masked from 'package:base':
##
##     attach, detach, load, save

## R.utils v2.13.0 (2025-02-24 21:20:02 UTC) successfully loaded. See ?R.utils for help.

##
## Attaching package: 'R.utils'

## The following object is masked from 'package:utils':
##
##     timestamp

## The following objects are masked from 'package:base':
##
##     cat, commandArgs, getOption, isOpen, nullfile, parse, use, warnings
```

```r
library(xts)
library(readxl)
library(fBasics)
```

```
##
## Attaching package: 'fBasics'

## The following objects are masked from 'package:R.oo':
##
##     getDescription, getTitle

## The following objects are masked from 'package:rugarch':
##
##     qgh, qnig

## The following object is masked from 'package:TTR':
##
##     volatility
```

```r
library(lmtest)
```

```
##
## Attaching package: 'lmtest'

## The following object is masked from 'package:R.utils':
##
##     reset
```

```r
library(sandwich)
library(fUnitRoots)
```

```
##
## Attaching package: 'fUnitRoots'

## The following objects are masked from 'package:urca':
##
##     punitroot, qunitroot, unitrootTable
```

```r
library(stats)
library(graphics)
library("MTS")
```

```
##
## Attaching package: 'MTS'

## The following object is masked from 'package:FinTS':
##
##     apca
```

Load the data from excel file, weekly spread

```r
rm(list= ls())
getwd()
```

```
## [1] "/Users/jacopo/Desktop"
```

```r
#setwd("~/Desktop")
data <- read_excel("Spread-weekly.xlsx")
```

The data is inverse, we need to change sign

```r
data$`IT10DE10=TWEB (BID)` <- as.numeric(data$`IT10DE10=TWEB (BID)`) * -1
```

Inspect data

```r
head(data)
```

```
## # A tibble: 6 x 2
##   Date                `IT10DE10=TWEB (BID)`
##   <dttm>                               <dbl>
## 1 2026-03-23 00:00:00                   93.2
## 2 2026-03-16 00:00:00                   90.8
## 3 2026-03-09 00:00:00                   81.4
## 4 2026-03-02 00:00:00                   77.3
## 5 2026-02-23 00:00:00                   63
## 6 2026-02-16 00:00:00                   60.4
```

```r
str(data)
```

```
## tibble [534 x 2] (S3: tbl_df/tbl/data.frame)
##  $ Date               : POSIXct[1:534], format: "2026-03-23" "2026-03-16" ...
##  $ IT10DE10=TWEB (BID): num [1:534] 93.2 90.8 81.4 77.3 63 60.4 65 66.2 65.7 64.4 ...
```

```r
sum(is.na(data))
```

```
## [1] 0
```

Parse the data

```r
library(zoo)
data$Date <- as.Date(data$Date)
spread <- zoo(data$`IT10DE10=TWEB (BID)`, order.by = data$Date)
spread
```

```
## 2016-01-04 2016-01-11 2016-01-18 2016-01-25 2016-02-01 2016-02-08 2016-02-15
##      101.8      108.2      116.4      115.9      125.6      140.7      138.5
## 2016-02-22 2016-02-29 2016-03-07 2016-03-14 2016-03-21 2016-03-29 2016-04-04
##      135.2      123.3      107.0      105.8      112.3      107.4      122.1
## 2016-04-11 2016-04-18 2016-04-25 2016-05-02 2016-05-09 2016-05-16 2016-05-23
##      121.9      124.5      121.9      133.7      136.1      131.8      122.2
## 2016-05-30 2016-06-06 2016-06-13 2016-06-20 2016-06-27 2016-07-04 2016-07-11
##      126.7      129.6      143.0      156.4      127.5      130.3      125.2
## 2016-07-18 2016-07-25 2016-08-01 2016-08-08 2016-08-15 2016-08-22 2016-08-29
##      132.9      134.6      127.0      122.3      118.0      119.7      121.6
## 2016-09-05 2016-09-12 2016-09-19 2016-09-26 2016-10-03 2016-10-10 2016-10-17
##      124.8      133.5      129.6      132.1      135.9      132.5      136.4
## 2016-10-24 2016-10-31 2016-11-07 2016-11-14 2016-11-21 2016-11-28 2016-12-05
##      142.0      154.8      166.2      173.5      185.8      163.8      169.5
## 2016-12-12 2016-12-19 2016-12-27 2017-01-02 2017-01-09 2017-01-16 2017-01-23
##      157.7      159.7      163.8      167.2      163.5      167.2      178.5
## 2017-01-30 2017-02-06 2017-02-13 2017-02-20 2017-02-27 2017-03-06 2017-03-13
##      184.7      193.5      187.2      199.3      176.8      186.9      193.0
## 2017-03-20 2017-03-27 2017-04-03 2017-04-10 2017-04-18 2017-04-24 2017-05-02
##      200.0      197.9      199.0      209.7      203.3      194.8      175.5
## 2017-05-08 2017-05-15 2017-05-22 2017-05-29 2017-06-05 2017-06-12 2017-06-19
##      185.7      176.4      176.9      198.0      182.8      170.3      165.8
## 2017-06-26 2017-07-03 2017-07-10 2017-07-17 2017-07-24 2017-07-31 2017-08-07
##      169.3      176.6      176.6      158.2      157.7      153.9      164.6
## 2017-08-14 2017-08-21 2017-08-28 2017-09-04 2017-09-11 2017-09-18 2017-09-25
##      162.1      172.3      169.7      164.6      164.1      165.3      171.1
## 2017-10-02 2017-10-09 2017-10-16 2017-10-23 2017-10-30 2017-11-06 2017-11-13
##      175.5      165.2      160.0      155.5      142.8      143.9      146.9
## 2017-11-20 2017-11-27 2017-12-04 2017-12-11 2017-12-18 2017-12-27 2018-01-02
##      146.2      139.2      133.8      151.7      151.9      157.1      156.0
## 2018-01-08 2018-01-15 2018-01-22 2018-01-29 2018-02-05 2018-02-12 2018-02-19
##      146.7      146.4      143.1      132.0      135.1      128.4      141.0
## 2018-02-26 2018-03-05 2018-03-12 2018-03-19 2018-03-26 2018-04-03 2018-04-09
##      140.0      134.7      140.0      135.5      129.7      129.2      129.0
## 2018-04-16 2018-04-23 2018-04-30 2018-05-07 2018-05-14 2018-05-21 2018-05-28
```

```
##      119.0      117.2      124.3      132.9      164.3      205.8      232.2
## 2018-06-04 2018-06-11 2018-06-18 2018-06-25 2018-07-02 2018-07-09 2018-07-16
##      269.3      220.9      236.5      237.5      243.5      229.0      228.1
## 2018-07-23 2018-07-30 2018-08-06 2018-08-13 2018-08-20 2018-08-27 2018-09-03
##      232.8      256.0      264.2      284.3      280.1      289.4      249.8
## 2018-09-10 2018-09-17 2018-09-24 2018-10-01 2018-10-08 2018-10-15 2018-10-22
##      253.2      236.6      267.6      285.4      305.7      311.1      311.6
## 2018-10-29 2018-11-05 2018-11-12 2018-11-19 2018-11-26 2018-12-03 2018-12-10
##      289.6      299.2      312.5      308.5      290.0      289.2      269.6
## 2018-12-17 2018-12-24 2018-12-31 2019-01-07 2019-01-14 2019-01-21 2019-01-28
##      257.7      250.3      268.8      268.3      252.4      250.5      264.1
## 2019-02-04 2019-02-11 2019-02-18 2019-02-25 2019-03-04 2019-03-11 2019-03-18
##      291.3      270.9      276.8      255.7      245.3      242.8      248.7
## 2019-03-25 2019-04-01 2019-04-08 2019-04-15 2019-04-23 2019-04-29 2019-05-06
##      256.0      248.2      233.4      256.1      262.2      253.7      273.6
## 2019-05-13 2019-05-20 2019-05-27 2019-06-03 2019-06-10 2019-06-17 2019-06-24
##      277.2      267.1      284.5      262.3      257.6      244.8      240.9
## 2019-07-01 2019-07-08 2019-07-15 2019-07-22 2019-07-29 2019-08-05 2019-08-12
##      209.9      198.3      198.4      199.2      207.7      239.1      209.6
## 2019-08-19 2019-08-26 2019-09-02 2019-09-09 2019-09-16 2019-09-23 2019-09-30
##      198.6      176.6      151.0      133.2      143.8      141.2      141.6
## 2019-10-07 2019-10-14 2019-10-21 2019-10-28 2019-11-04 2019-11-11 2019-11-18
##      138.4      130.8      132.8      137.8      144.7      157.7      155.2
## 2019-11-25 2019-12-02 2019-12-09 2019-12-16 2019-12-23 2019-12-30 2020-01-06
##      159.3      163.7      146.0      157.5      153.7      152.8      156.5
## 2020-01-13 2020-01-20 2020-01-27 2020-02-03 2020-02-10 2020-02-17 2020-02-24
##      163.6      159.4      140.2      136.8      135.4      134.8      172.0
## 2020-03-02 2020-03-09 2020-03-16 2020-03-23 2020-03-30 2020-04-06 2020-04-14
##      180.1      241.8      193.9      182.2      197.5      195.2      229.3
## 2020-04-20 2020-04-27 2020-05-04 2020-05-11 2020-05-18 2020-05-25 2020-06-01
##      236.3      236.8      234.0      238.6      208.9      195.5      169.3
## 2020-06-08 2020-06-15 2020-06-22 2020-06-29 2020-07-06 2020-07-13 2020-07-20
##      186.6      178.0      181.2      174.5      174.3      167.6      144.9
## 2020-07-27 2020-08-03 2020-08-10 2020-08-17 2020-08-24 2020-08-31 2020-09-07
##      154.8      143.2      141.9      145.4      144.8      150.4      146.8
## 2020-09-14 2020-09-21 2020-09-28 2020-10-05 2020-10-12 2020-10-19 2020-10-26
##      140.1      136.7      127.6      121.0      122.8      134.3      134.2
## 2020-11-02 2020-11-09 2020-11-16 2020-11-23 2020-11-30 2020-12-07 2020-12-14
##      124.6      117.6      118.6      115.0      113.9      116.1      111.2
## 2020-12-21 2020-12-28 2021-01-04 2021-01-11 2021-01-18 2021-01-25 2021-02-01
##      114.5      116.6      108.7      119.0      128.7      116.0       98.5
## 2021-02-08 2021-02-15 2021-02-22 2021-03-01 2021-03-08 2021-03-15 2021-03-22
##       91.4       93.6      103.0      106.0       94.0       96.6       96.1
## 2021-03-29 2021-04-06 2021-04-12 2021-04-19 2021-04-26 2021-05-03 2021-05-10
##       95.7      103.7      102.1      104.7      107.3      114.3      116.4
## 2021-05-17 2021-05-24 2021-05-31 2021-06-07 2021-06-14 2021-06-21 2021-06-28
##      112.2      105.7      103.4       97.2      107.3      107.4      101.5
## 2021-07-05 2021-07-12 2021-07-19 2021-07-26 2021-08-02 2021-08-09 2021-08-16
##      106.0      105.1      104.5      108.4      102.8      101.9      104.2
## 2021-08-23 2021-08-30 2021-09-06 2021-09-13 2021-09-20 2021-09-27 2021-10-04
##      107.0      107.2      104.4      100.7      101.4      104.2      104.2
## 2021-10-11 2021-10-18 2021-10-25 2021-11-01 2021-11-08 2021-11-15 2021-11-22
##      105.3      106.3      125.5      111.6      122.6      121.1      130.7
## 2021-11-29 2021-12-06 2021-12-13 2021-12-20 2021-12-27 2022-01-03 2022-01-10
```

```
##      131.8      131.9      129.8      137.6      136.0      140.3      135.9
## 2022-01-17 2022-01-24 2022-01-31 2022-02-07 2022-02-14 2022-02-21 2022-02-28
##      135.8      134.6      154.9      167.1      165.7      163.5      164.1
## 2022-03-07 2022-03-14 2022-03-21 2022-03-28 2022-04-04 2022-04-11 2022-04-19
##      160.8      152.2      151.6      155.7      166.9      165.9      171.0
## 2022-04-25 2022-05-02 2022-05-09 2022-05-16 2022-05-23 2022-05-30 2022-06-06
##      184.1      201.0      190.9      204.7      195.8      214.3      227.8
## 2022-06-13 2022-06-20 2022-06-27 2022-07-04 2022-07-11 2022-07-18 2022-07-25
##      192.4      200.2      187.9      198.7      219.6      235.8      231.7
## 2022-08-01 2022-08-08 2022-08-15 2022-08-22 2022-08-29 2022-09-05 2022-09-12
##      203.6      207.7      217.0      222.9      234.6      232.7      229.2
## 2022-09-19 2022-09-26 2022-10-03 2022-10-10 2022-10-17 2022-10-24 2022-10-31
##      229.3      244.5      250.7      245.4      234.7      207.1      217.9
## 2022-11-07 2022-11-14 2022-11-21 2022-11-28 2022-12-05 2022-12-12 2022-12-19
##      205.0      187.6      189.4      191.7      188.9      213.0      212.9
## 2022-12-27 2023-01-02 2023-01-09 2023-01-16 2023-01-23 2023-01-30 2023-02-06
##      213.8      199.3      186.1      184.5      200.3      185.7      184.5
## 2023-02-13 2023-02-20 2023-02-27 2023-03-06 2023-03-13 2023-03-20 2023-03-27
##      185.4      190.7      183.7      182.7      195.1      188.0      182.2
## 2023-04-03 2023-04-11 2023-04-17 2023-04-24 2023-05-02 2023-05-08 2023-05-15
##      184.7      186.5      187.4      187.7      191.1      190.7      183.3
## 2023-05-22 2023-05-29 2023-06-05 2023-06-12 2023-06-19 2023-06-26 2023-07-03
##      186.2      166.6      173.4      156.2      164.3      168.1      173.1
## 2023-07-10 2023-07-17 2023-07-24 2023-07-31 2023-08-07 2023-08-14 2023-08-21
##      170.1      164.6      165.4      168.2      163.3      170.7      167.9
## 2023-08-28 2023-09-04 2023-09-11 2023-09-18 2023-09-25 2023-10-02 2023-10-09
##      167.8      173.9      178.9      184.2      194.1      204.9      202.9
## 2023-10-16 2023-10-23 2023-10-30 2023-11-06 2023-11-13 2023-11-20 2023-11-27
##      202.9      197.0      180.7      180.7      171.8      169.3      169.5
## 2023-12-04 2023-12-11 2023-12-18 2023-12-27 2024-01-02 2024-01-08 2024-01-15
##      174.0      166.3      153.7      164.4      163.4      153.8      157.2
## 2024-01-22 2024-01-29 2024-02-05 2024-02-12 2024-02-19 2024-02-26 2024-03-04
##      155.6      158.0      158.2      148.1      144.8      146.1      130.4
## 2024-03-11 2024-03-18 2024-03-25 2024-04-02 2024-04-08 2024-04-15 2024-04-22
##      126.7      131.1      137.1      139.1      138.8      139.4      131.8
## 2024-04-29 2024-05-06 2024-05-13 2024-05-20 2024-05-27 2024-06-03 2024-06-10
##      128.9      130.0      125.7      127.0      128.3      130.3      153.6
## 2024-06-17 2024-06-24 2024-07-01 2024-07-08 2024-07-15 2024-07-22 2024-07-29
##      150.6      155.6      135.7      128.3      129.1      134.6      145.3
## 2024-08-05 2024-08-12 2024-08-19 2024-08-26 2024-09-02 2024-09-09 2024-09-16
##      140.7      139.0      134.5      140.3      137.3      129.5      127.8
## 2024-09-23 2024-09-30 2024-10-07 2024-10-14 2024-10-21 2024-10-28 2024-11-04
##      123.9      122.8      121.1      109.3      121.7      127.6      129.3
## 2024-11-11 2024-11-18 2024-11-25 2024-12-02 2024-12-09 2024-12-16 2024-12-23
##      119.8      125.5      120.4      109.3      113.7      116.9      114.8
## 2024-12-30 2025-01-06 2025-01-13 2025-01-20 2025-01-27 2025-02-03 2025-02-10
##      117.2      120.5      111.4      108.9      109.5      109.3      109.7
## 2025-02-17 2025-02-24 2025-03-03 2025-03-10 2025-03-17 2025-03-24 2025-03-31
##      109.5      113.9      111.9      112.7      112.1      112.1      120.3
## 2025-04-07 2025-04-14 2025-04-22 2025-04-28 2025-05-05 2025-05-12 2025-05-19
##      126.6      118.3      112.1      109.9      105.4      100.5      102.5
## 2025-05-26 2025-06-02 2025-06-09 2025-06-16 2025-06-23 2025-06-30 2025-07-07
##       98.8       93.0       95.6       98.9       88.9       87.0       87.7
## 2025-07-14 2025-07-21 2025-07-28 2025-08-04 2025-08-11 2025-08-18 2025-08-25
```

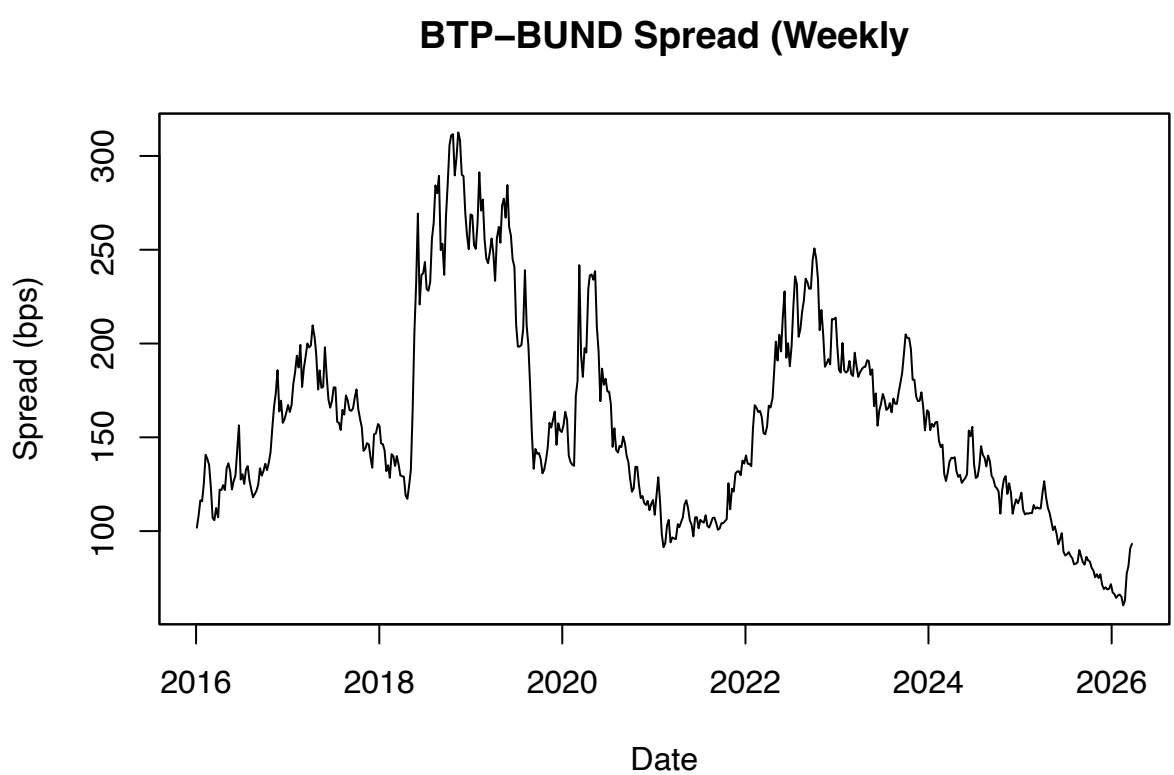
```
##       88.8       86.8       85.5       82.3       82.6       83.4       89.9
## 2025-09-01 2025-09-08 2025-09-15 2025-09-22 2025-09-29 2025-10-06 2025-10-13
##       86.7       83.4       82.1       86.3       84.4       83.8       80.5
## 2025-10-20 2025-10-27 2025-11-03 2025-11-10 2025-11-17 2025-11-24 2025-12-01
##       79.0       75.4       77.0       75.0       77.0       71.4       69.1
## 2025-12-08 2025-12-15 2025-12-22 2025-12-29 2026-01-05 2026-01-12 2026-01-19
##       70.0       68.9       69.3       71.7       67.3       66.4       64.4
## 2026-01-26 2026-02-02 2026-02-09 2026-02-16 2026-02-23 2026-03-02 2026-03-09
##       65.7       66.2       65.0       60.4       63.0       77.3       81.4
## 2026-03-16 2026-03-23
##       90.8       93.2
```

```r
any(is.infinite(spread))
```

```
## [1] FALSE
```

Plot the data

```r
plot(spread, main="BTP-BUND Spread (Weekly", ylab="Spread (bps)", xlab= "Date")
```




```r
options(scipen = 999)
basicStats(spread)
```

```
##                        x
```

```
## nobs          534.000000
## NAs             0.000000
## Minimum        60.400000
## Maximum       312.500000
## 1. Quartile   121.025000
## 3. Quartile   187.675000
## Mean          158.577715
## Median        150.500000
## Sum         84680.500000
## SE Mean         2.298415
## LCL Mean      154.062653
## UCL Mean      163.092778
## Variance     2820.967026
## Stdev          53.112776
## Skewness        0.674180
## Kurtosis       -0.004433
```

Skewness (Asimmetria) positiva: 1.53 Un valore di 1.53 è alto. Significa che la distribuzione ha una coda lunga verso destra.

Cosa ti dice: La maggior parte delle volte lo spread è "basso" (vicino alla mediana di 158), ma ci sono stati eventi estremi (il massimo a 532) che tirano la media verso l'alto (184).

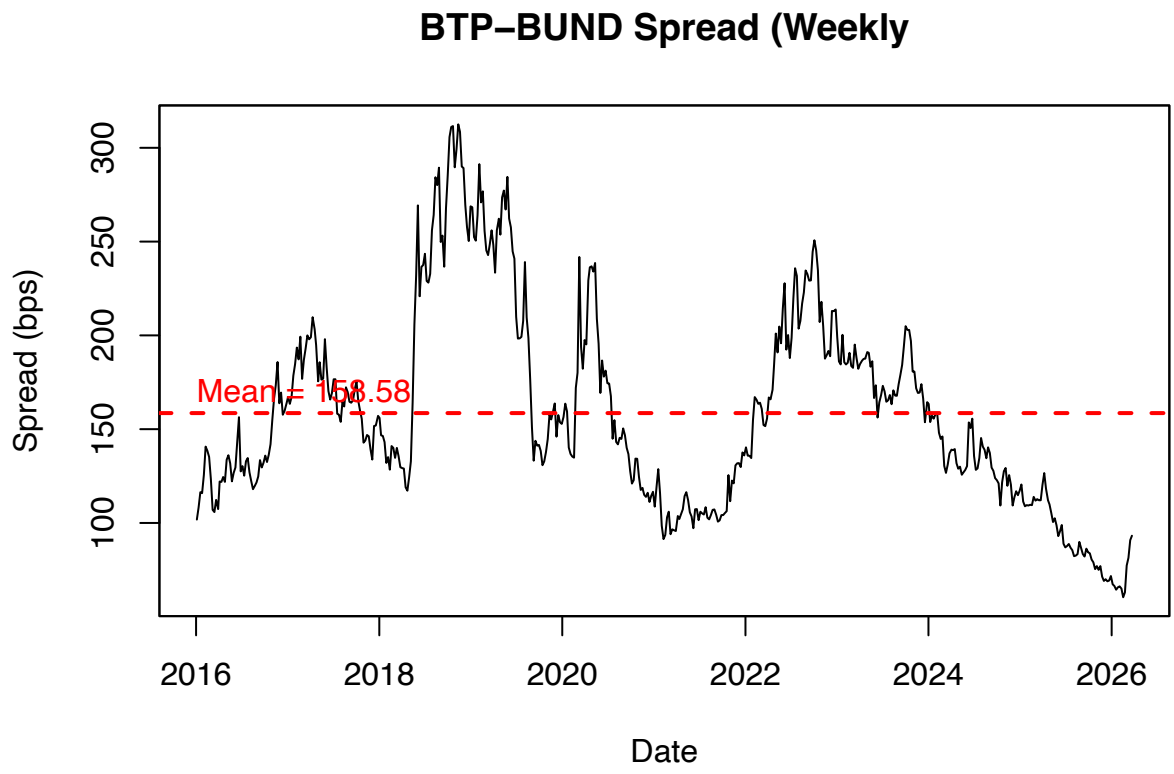
Forecast 1

(using Naive Forecast (Mean))

```r
mean <- mean(data$`IT10DE10=TWEB (BID)`, na.rm=TRUE)
```

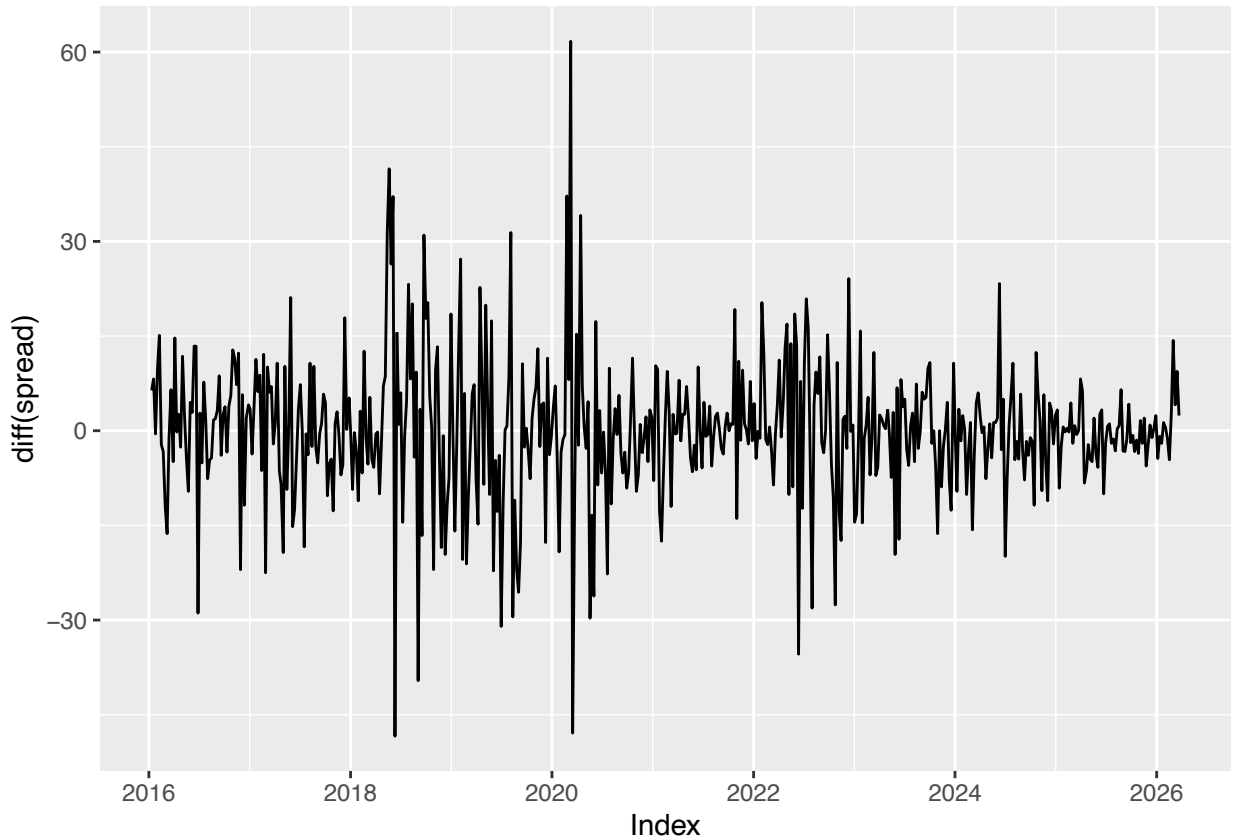
Plot spread values with the mean value

```r
plot(spread, main="BTP-BUND Spread (Weekly", ylab="Spread (bps)", xlab= "Date")
abline(h = mean, col="red", lwd=2, lty=2)
text(x = start(spread), y= mean,
     labels=paste("Mean =", round(mean, 2)),
     col="red", adj= c(0, -0.5))
```

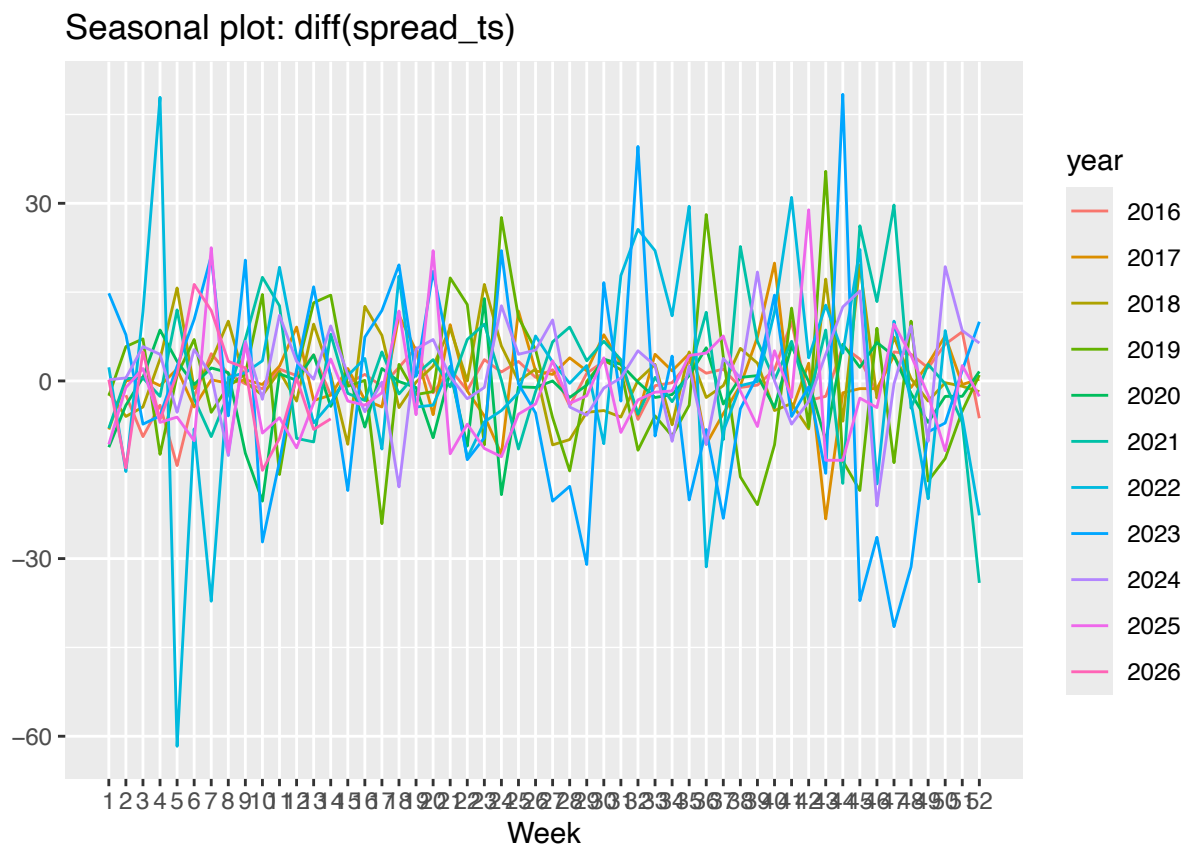


Adding a visualization for seasonality We plot the season using ggplot2 library, using 2 methods

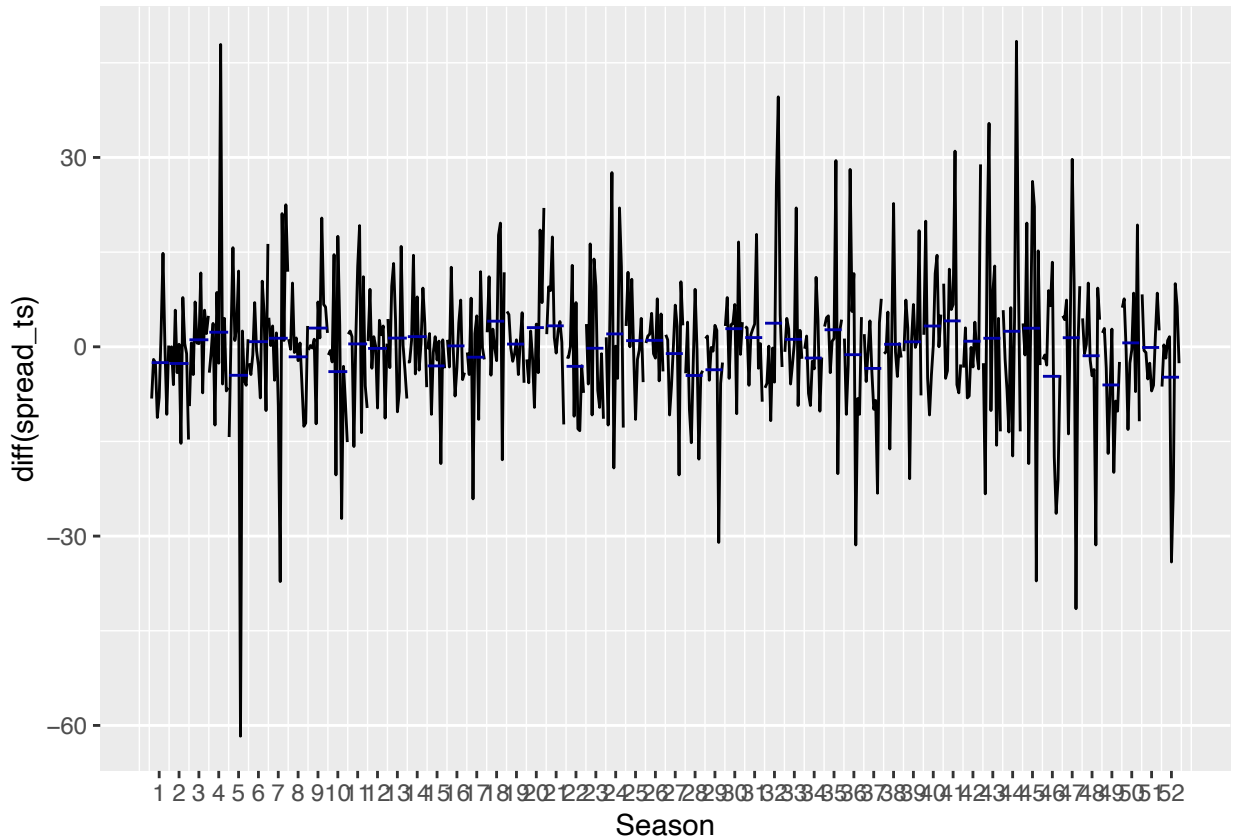
```
library(forecast)
library(ggplot2)

autoplot(diff(spread))
```



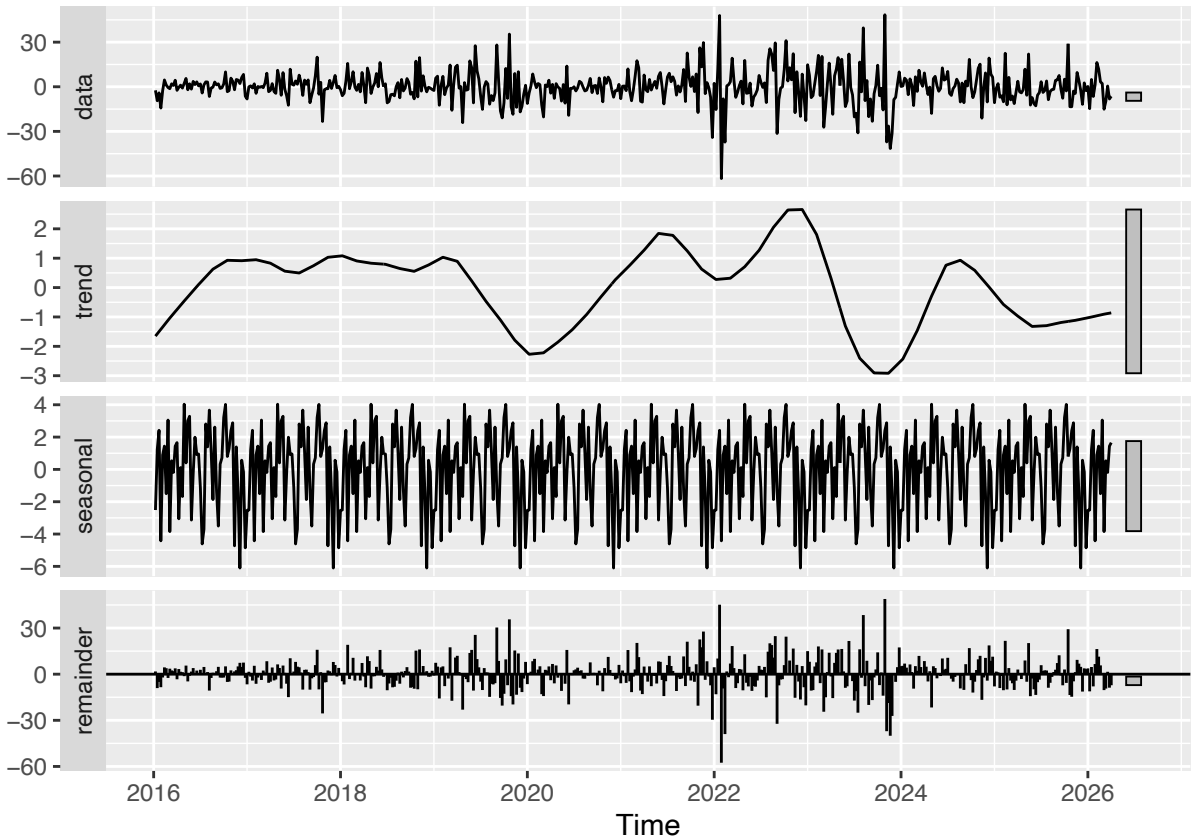

```
#To use this method, we need a ts object, let's create one
spread_ts <- ts(data$`IT10DE10=TWEB (BID)`, start=c(2016, 1), frequency = 52)
ggseasonplot(diff(spread_ts))
```




```
ggsubseriesplot(diff(spread_ts))
```




```
#Look also at decomposed series
decomposed <- stl(diff(spread_ts), s.window="periodic")
autoplot(decomposed)
```




```r
##Find the ratio between seasonal and data range, if < 15% we ignore seasonality
decomp <- stl(spread_ts, s.window = "periodic")

seasonal_component <- decomp$time.series[, "seasonal"]
data_range    <- diff(range(spread_ts))
seasonal_range <- diff(range(seasonal_component))

ratio <- seasonal_range / data_range
cat("Seasonal ratio:", round(ratio * 100, 2), "%\n")
```

```
## Seasonal ratio: 8.47 %
```

```r
##We get 8.47%, so we ignore seasonality
```

Formal test for seasonality in R

We use seastests package

```r
#install.packages("seastests")
library(seastests)

isSeasonal(spread_ts, test="combined")
```

```
## [1] FALSE
```

```r
#We get FALSE

#Individual tests
isSeasonal(spread_ts, test="qs")
```

```
## [1] FALSE
```

```r
isSeasonal(spread_ts, test="seasdum")
```

```
## [1] FALSE
```

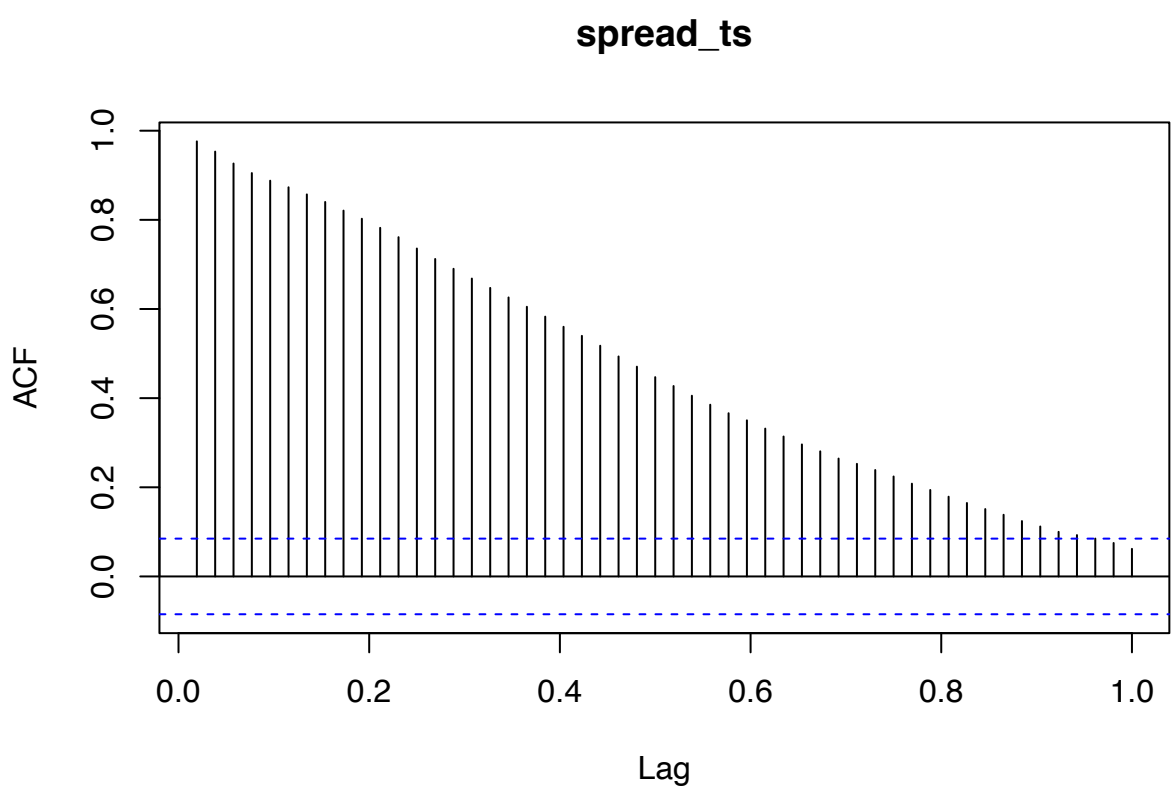
```r
isSeasonal(spread_ts, test="kw")
```

```
## [1] FALSE
```

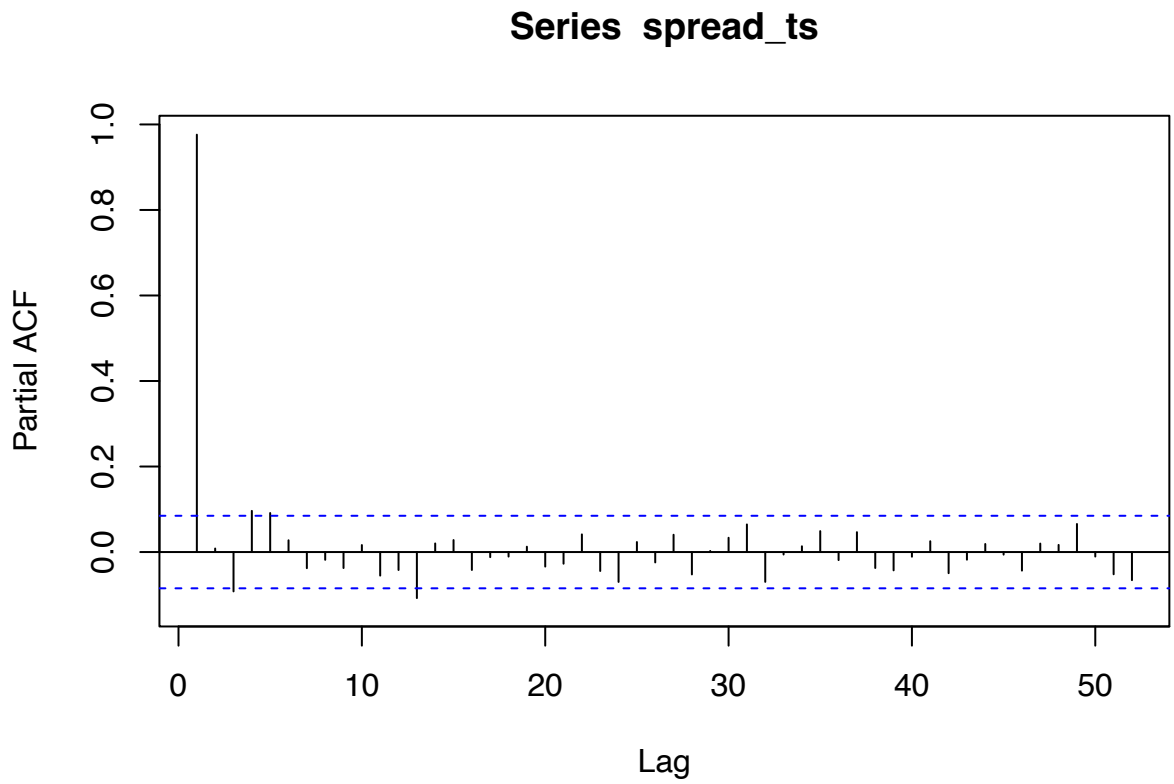
```r
isSeasonal(spread_ts, test="fried")
```

```
## [1] FALSE
```

```r
#We get all FALSE

## Now plot ACF and PACF
Acf(spread_ts, lag.max= 52)
```



```r
Pacf(spread_ts, lag.max = 52)
```




```r
#We have 1 significant lag at 1 in the Pacf, and a Acf which gradually goes to 0
#This suggests no seasonality.

library(tseries)
adf.test(spread_ts)     # stationarity test -> we have p-value = 0.546, so we fail to reject the null,
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  spread_ts
## Dickey-Fuller = -2.0753, Lag order = 8, p-value = 0.5464
## alternative hypothesis: stationary
```

```r
kpss.test(spread_ts) # stationarity test where the null hypothesis is that the series IS stationary
```

```
## Warning in kpss.test(spread_ts): p-value smaller than printed p-value

##
##  KPSS Test for Level Stationarity
##
## data:  spread_ts
## KPSS Level = 1.4359, Truncation lag parameter = 6, p-value = 0.01
```

```
# p-value is 0.01 -> we REJECT the null, so the series is NOT seasonal.

## Finally, we can affirm that our data does not contain seasonality, so we can work using normal ARIMA
```

Forecast 1.1

(using ARIMA model)

Check for stationarity

```
library(fUnitRoots)
library(urca)
```

Better to work on log(spread)? NO: (MA DA VERIFICARE)
"Following the standard econometric literature (e.g., Campbell, Lo, and MacKinlay, 1997; Hamilton, 1994), the BTP-BUND spread is analyzed in levels rather than logarithmic returns. As noted by Engle (1982) in his foundational work on GARCH models, interest rate differentials are already expressed in percentage terms. Applying a log-transformation would lead to numerical instability as values approach the zero-lower bound and would obscure the direct economic interpretability of the results in basis points."

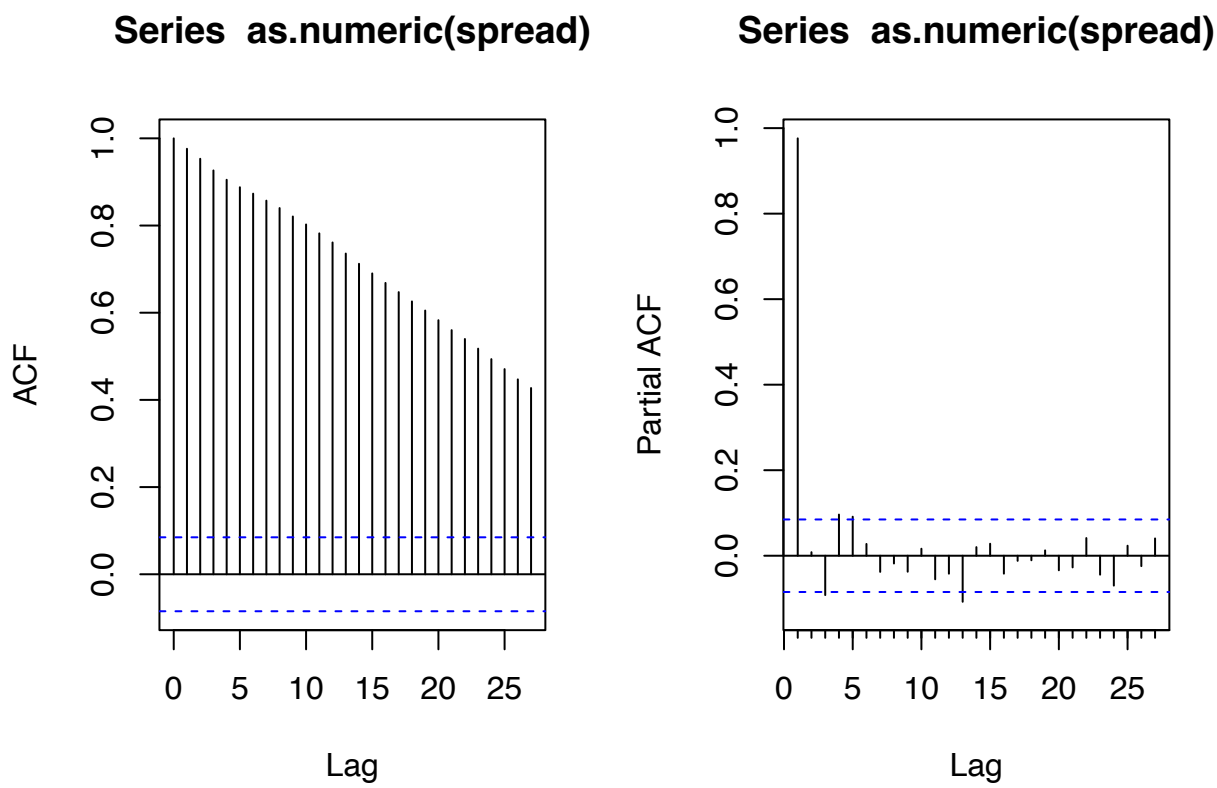
Oltre ai manuali classici (Hamilton, Campbell), se vuoi citare dei paper accademici specifici che trattano lo spread sovrano (BTP-BUND o simili) e i modelli GARCH senza ricorrere ai logaritmi, ecco i tre più autorevoli.

Questi articoli sono perfetti per la tua bibliografia perché affrontano esattamente il tuo tema: rischio sovrano, volatilità e spread.

ome sintetizzare queste fonti nella tua tesi (English) Literature Review on Model Specification:

"The choice to model the BTP-BUND spread in first differences of its levels is supported by the foundational empirical work on Eurozone sovereign spreads. Codogno, Favero, and Missale (2003) and Favero et al. (2010) demonstrate that interest rate differentials are best captured in basis points to maintain economic relevance and linear properties between credit risk and liquidity factors. Furthermore, the application of a GARCH framework, as pioneered by Bollerslev (1986), ensures that the time-varying volatility of the spread is explicitly parameterized rather than suppressed via logarithmic transformations."

```
par(mfrow = c(1, 2))
acf(as.numeric(spread))
Pacf(as.numeric(spread))
```




```r
par(mfrow = c(1, 1))
```

The very slow decay of acf indicates a "quasi - unit" root of AR component

(CHIEDERE ALLA PROF COME DECIDERE IL NUMERO DI LAG, NEL DUBBIO USO UNA FUNZIONE CHE DECIDE AUTONOMAMENTW QUANTI USARNE TRAMITE BASANDOSI SULL' AIC/BIC)

We now execute the ADF test to check for Unit root To ensure a robust analysis, I performed the Augmented Dickey-Fuller (ADF) test using two different specifications: one including only a constant (drift) and one including both a constant and a linear time trend.

```r
risultato_adf <- adfTest(spread, lags = 1, type = "c")
print(risultato_adf)
```

```
##
## Title:
##  Augmented Dickey-Fuller Test
##
## Test Results:
##   PARAMETER:
##     Lag Order: 1
##   STATISTIC:
##     Dickey-Fuller: -2.2975
##   P VALUE:
##     0.2019
```

```
##
## Description:
##  Fri Mar 27 21:20:20 2026 by user:
```

```r
# Equivalent with auto determination of lags
test_auto_drift <- ur.df(spread, type = "drift", lags = 10, selectlags = "AIC")
# type="drft" include intercetp (no trend)
summary(test_auto_drift)
```

```
##
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression drift
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 + 1 + z.diff.lag)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -41.774  -5.063  -0.411   5.039  58.274
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.707284   1.531927   1.767  0.07778 .
## z.lag.1     -0.017267   0.009127  -1.892  0.05907 .
## z.diff.lag1 -0.018597   0.043813  -0.424  0.67141
## z.diff.lag2  0.100953   0.043341   2.329  0.02023 *
## z.diff.lag3 -0.118101   0.043402  -2.721  0.00673 **
## z.diff.lag4 -0.108814   0.043768  -2.486  0.01323 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 10.83 on 517 degrees of freedom
## Multiple R-squared:  0.04435,    Adjusted R-squared:  0.03511
## F-statistic: 4.799 on 5 and 517 DF,  p-value: 0.0002683
##
##
## Value of test-statistic is: -1.8919 1.7949
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau2 -3.43 -2.86 -2.57
## phi1  6.43  4.59  3.78
```

Stationarity Analysis: **ADF Test Verdict** To determine if the spread is stationary, we must compare the Value of test-statistic (τ_2) against the Critical Values.
Test Statistic (τ_2): -2.3395
Critical Values:1%: -3.43;5%: -2.86;10%: -2.57
Result: Since -2.3395 is greater (less negative) than all the critical values, we fail to reject the null hypothesis (H_0).

Conclusion: The spread is non-stationary. It possesses a "unit root," meaning it tends to wander and does not quickly revert to its mean. In terms of trading or financial analysis, this spread exhibits "memory" and is not (yet) suitable for pure mean-reversion strategies in its current form.

The Second Value: 2.7426 (ϕ_1) –> This is a joint F-test.
In simple terms, it tests the null hypothesis that the series has a "unit root" AND that there is "no intercept" (drift) at the same time.
Purpose: It helps you determine if the model specification you chose (type = "drift") is correct or if a simpler model without an intercept would have been sufficient.
Comparison: This value must be compared against the ϕ_1 row in the critical values table:1%: 6.43;5%: 4.59;10%: 3.78
Interpretation in Your Case: Your value (2.7426) is lower than all the critical values, this means we fail to reject the joint null hypothesis. This suggests that the intercept (the "drift" term) is not statistically strong in this specific sample. This further confirms that the BTP-BUND spread is currently in a phase of structural uncertainty, behaving more like a random walk without a clear, constant directional pull (drift).

Test with Trend (type = "trend"): In this model, we account for the possibility that the spread follows a consistent path over time (slope)

```r
risultato_adf <- adfTest(spread, lags = 1, type = "c")
print(risultato_adf)
```

```
##
## Title:
##  Augmented Dickey-Fuller Test
##
## Test Results:
##   PARAMETER:
##     Lag Order: 1
##   STATISTIC:
##     Dickey-Fuller: -2.2975
##   P VALUE:
##     0.2019
##
## Description:
##  Fri Mar 27 21:20:20 2026 by user:
```

```r
#Equivalent but with autodetermination of lags:
test_auto_drift <- ur.df(spread, type = "trend", lags = 10, selectlags = "AIC")
# type="trend" include both trend and intercept
summary(test_auto_drift)
```

```
##
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression trend
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 + 1 + tt + z.diff.lag)
##
## Residuals:
```

```
##     Min      1Q  Median      3Q     Max
## -41.624  -5.211  -0.226   4.882  58.057
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  5.474083   2.148614   2.548  0.01113 *
## z.lag.1     -0.024034   0.009827  -2.446  0.01479 *
## tt          -0.006205   0.003387  -1.832  0.06750 .
## z.diff.lag1 -0.018257   0.043714  -0.418  0.67638
## z.diff.lag2  0.101510   0.043243   2.347  0.01928 *
## z.diff.lag3 -0.116687   0.043310  -2.694  0.00729 **
## z.diff.lag4 -0.108408   0.043669  -2.482  0.01336 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 10.81 on 516 degrees of freedom
## Multiple R-squared:  0.05053,    Adjusted R-squared:  0.03949
## F-statistic: 4.577 on 6 and 516 DF,  p-value: 0.000154
##
##
## Value of test-statistic is: -2.4458 2.321 3.4762
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau3 -3.96 -3.41 -3.12
## phi2  6.09  4.68  4.03
## phi3  8.27  6.25  5.34
```

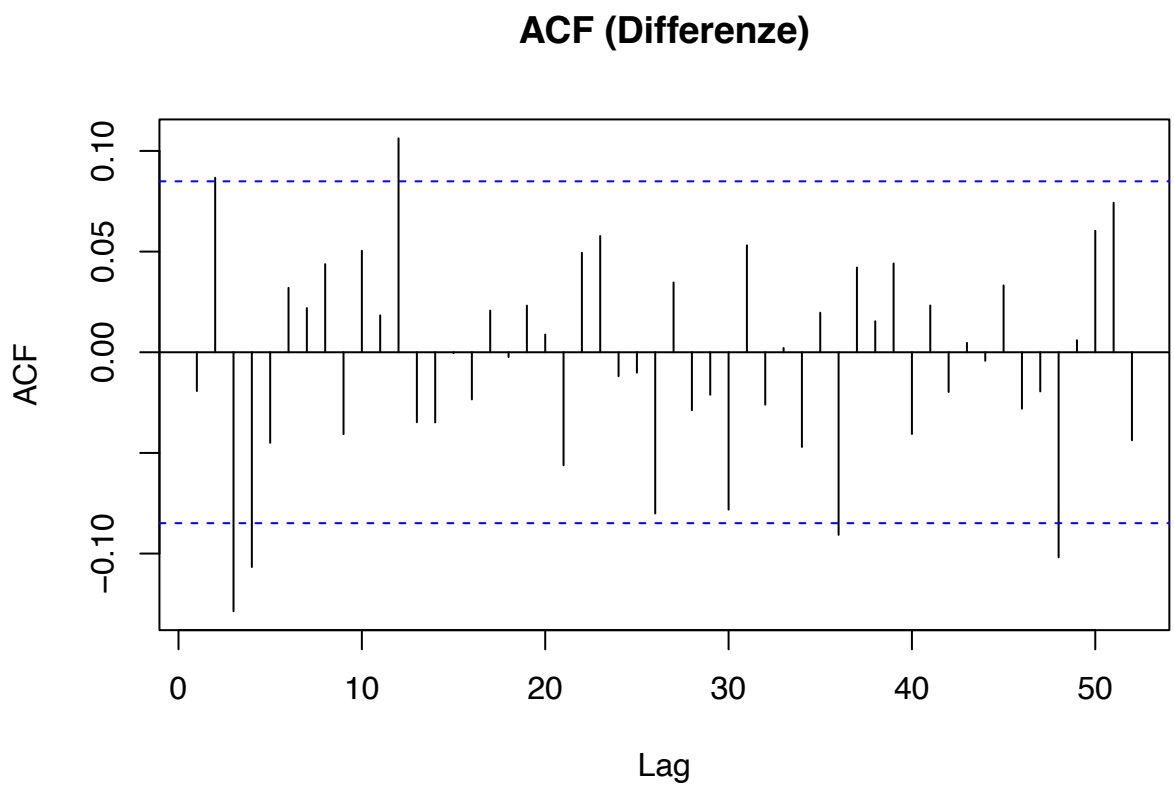
1) Test Statistic (τ_3): -3.0765
   Critical Value (5%): -3.41Verdict: Even with the inclusion of a trend component (which appeared significant with a p-value of 0.035), the test statistic -3.0765 is still above the 5% critical value of -3.41. We still fail to reject the null hypothesis,o the series is non-stationary.

2) phi2 (ϕ_2):
   The Triple Joint TestThis is the most restrictive test. It tests the null hypothesis that:There is a unit root AND There is no trend AND There is no drift (intercept).Your Result: 3.3214Critical Value (5%): 4.68
   Interpretation: Again, we fail to reject. This suggests the model might be "over-specified"—the spread is behaving so much like a Random Walk that adding the trend and intercept doesn't significantly improve the statistical fit of the unit root test.

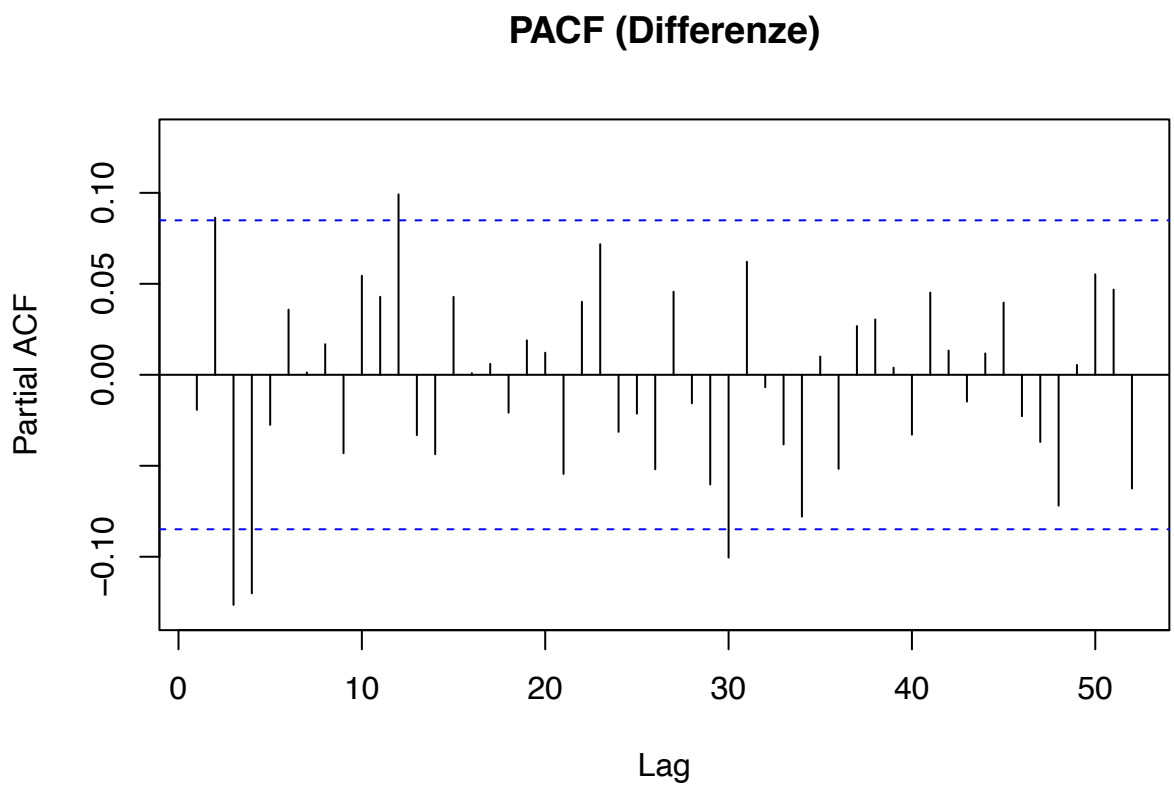
3) phi3 (ϕ_3): The Joint Test (Trend + Unit Root)
   This statistic performs a joint test on two parameters.It tests the null hypothesis that:There is a unit root ($\gamma = 0$) AND There is no time trend ($\beta = 0$).
   Your Result: 4.9762Critical Value (5%): 6.25
   Interpretation: Since $4.9762 < 6.25$, we fail to reject this joint null hypothesis.This means that statistically, we cannot say for sure that a trend actually exists once we account for the non-stationarity of the spread.

SO NOW DIFFERENTIATE THE SERIE AND CHECK FOR STATIONARITY

```r
# 1. Calcolo della differenza prima (Weekly Change)
diff_spread <- diff(spread)
par(mfrow = c(1, 1))
Acf(as.numeric(diff_spread), main="ACF (Differenze)", lag.max = 52)
```




```
Pacf(as.numeric(diff_spread), main="PACF (Differenze)", lag.max = 52)
```



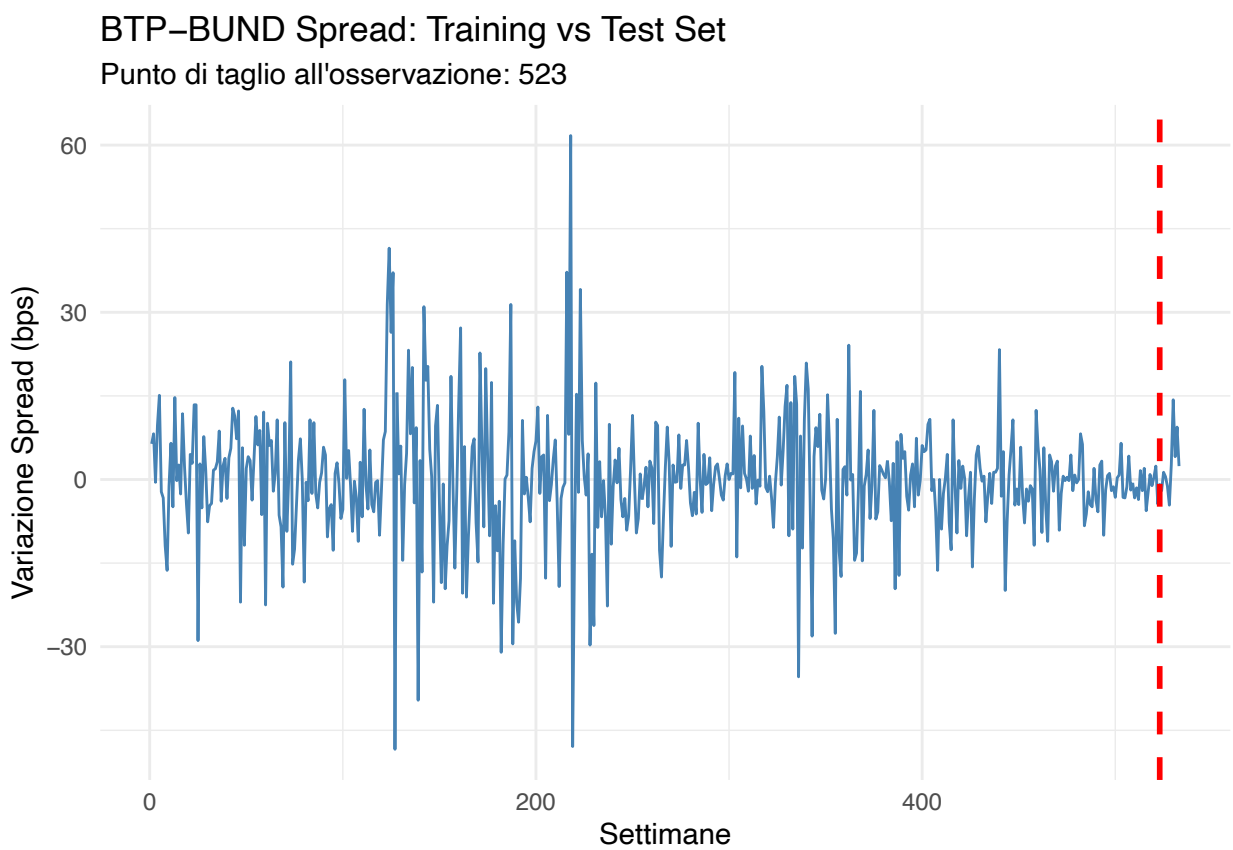

```
par(mfrow = c(1, 1))
```

–> ACF shows significant lags 3,4 and 12 –> PACF shows significant lags 2,3,4,12 HOWERER we first divide the dataset in train and test sets in order to evaluate tboth in-sample and out-sample of the ARIMA that we are going to estimate (PENSO SAREBBE PIU CORRETTO FARE QUESTA DIVISIONE ANCHE NELLA PARTE PRECEDENTE IN CUI SI TESTA LA STAZIONARIETA' DELLA SERIA NON DIFFERENZIATA, ANCHE SE NON VA A INFLUENZARE LE NOSTRE FORECAST)

```
n_test  <- 10
n_train <- length(diff_spread) - n_test

train_2 <- diff_spread[417:n_train]
train <- diff_spread[1:n_train]
test  <- diff_spread[(n_train + 1):length(diff_spread)]
library(ggplot2)

# Creiamo un piccolo data frame per il plot
df_plot <- data.frame(
  Settimana = 1:length(diff_spread),
  Valore = as.numeric(diff_spread)
)

ggplot(df_plot, aes(x = Settimana, y = Valore)) +
  geom_line(color = "steelblue") +
  # Questa è la tua linea verticale
```

```r
  geom_vline(xintercept = n_train, color = "red", linetype = "dashed", size = 1) +
  labs(title = "BTP-BUND Spread: Training vs Test Set",
       subtitle = paste("Punto di taglio all'osservazione:", n_train),
       x = "Settimane", y = "Variazione Spread (bps)") +
  theme_minimal()
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```




```r
cat(sprintf("Training observations: %d\n", n_train))
```

```
## Training observations: 523
```

```r
cat(sprintf("Test observations:     %d\n", n_test))
```

```
## Test observations:     10
```

```r
cat(sprintf("Train period: %s -> %s\n", index(train)[1], index(train)[length(train)]))
```

```
## Train period: 2016-01-11 -> 2026-01-12
```

```r
cat(sprintf("Test period:  %s -> %s\n", index(test)[1],  index(test)[length(test)]))
```

```
## Test period:  2026-01-19 -> 2026-03-23
```

IS MEAN DIFFERENT FROM 0?

```r
t.test(train)
```

```
##
##  One Sample t-test
##
## data:  train
## t = -0.1398, df = 522, p-value = 0.8889
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  -1.0188718  0.8834989
## sample estimates:
##   mean of x
## -0.06768642
```

NO, mean(diff_spread) in train data is NOT statistically different from 0

Test ADF sulle differenze per confermare la stazionarietà Usiamo type = "none" perché la differenziazione rimuove il drift: Se lo spread ha un trend (es. cresce costantemente), la sua differenza prima (ΔSpread) avrà una media costante positiva (un drift).
Se invece ipotizzassi un trend anche sulla differenza prima, significherebbe che lo spread non sta solo crescendo, ma sta accelerando (es. ogni settimana aumenta di 1 punto base in più rispetto alla settimana precedente).
In economia, i tassi d'interesse e gli spread sono considerati **"mean-reverting"** o comunque legati a fondamentali; l'idea che accelerino all'infinito verso l'alto o verso il basso non ha senso economico nel lungo periodo.

1. Paper Fondamentali (The "Must-Read") Questi tre lavori spiegano perché gli spread e i tassi tornano sempre "a casa":

Vasicek (1977): An Equilibrium Characterization of the Term Structure of Interest Rates

Perché è importante: È il primo modello a introdurre matematicamente la mean-reversion. Vasicek dimostra che se i tassi si allontanano troppo da un valore di equilibrio, una "forza" li riporta indietro.

Applicazione: Giustifica perché non usi il trend nel tuo test ADF.

Cox, Ingersoll, & Ross (1985): A Theory of the Term Structure of Interest Rates

Perché è importante: Evoluzione di Vasicek. Introducono il concetto che la volatilità dipende dal livello del tasso (se lo spread è alto, è più instabile).

Applicazione: Questo è il "nonno" teorico del tuo modello GARCH.

Balduzzi, Das, & Foresi (1998): The Central Tendency: A Second Factor in Bond Yields

Perché è importante: Analizza come i tassi d'interesse oscillino attorno a una "tendenza centrale" che cambia lentamente, legata ai fondamentali macroeconomici.

```r
test_stationarity_diff<-adfTest(train,lags=1,type="nc")
```

```
## Warning in adfTest(train, lags = 1, type = "nc"): p-value smaller than printed
## p-value
```

```r
print(test_stationarity_diff)
```

```
##
## Title:
##  Augmented Dickey-Fuller Test
##
## Test Results:
##   PARAMETER:
##     Lag Order: 1
##   STATISTIC:
##     Dickey-Fuller: -14.9696
##   P VALUE:
##     0.01
##
## Description:
##  Fri Mar 27 21:20:20 2026 by user:
```

```r
#equivalent but with auto determination of lags
test_stationarity_diff_autolags <- ur.df(train, type = "none", lags = 10, selectlags = "AIC")

summary(test_stationarity_diff_autolags)
```

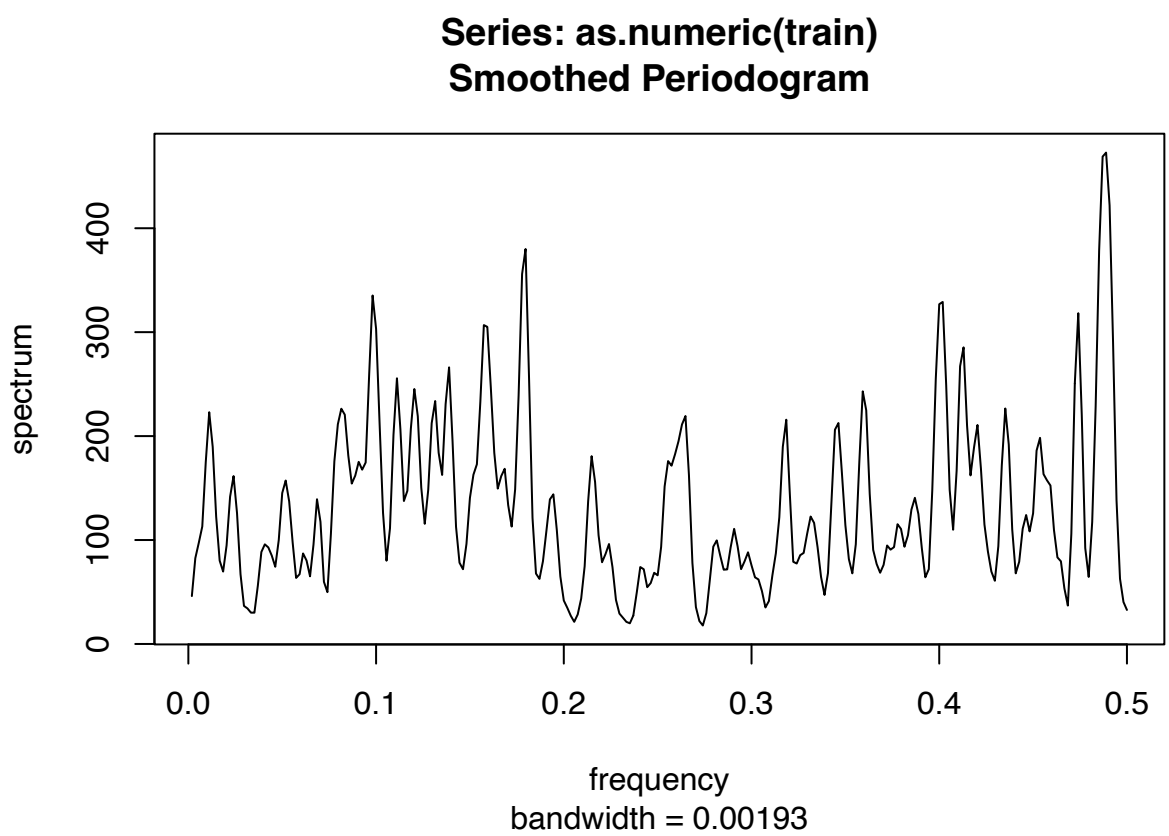
```
##
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression none
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 - 1 + z.diff.lag)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -42.103  -5.083  -0.182   4.790  58.250
##
## Coefficients:
##             Estimate Std. Error t value             Pr(>|t|)
## z.lag.1     -1.18315    0.08797 -13.449 < 0.0000000000000002 ***
## z.diff.lag1  0.15273    0.07458   2.048               0.0411 *
## z.diff.lag2  0.24558    0.06224   3.946            0.0000908 ***
## z.diff.lag3  0.11807    0.04391   2.689               0.0074 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 10.93 on 508 degrees of freedom
```

```
## Multiple R-squared:  0.5324, Adjusted R-squared:  0.5287
## F-statistic: 144.6 on 4 and 508 DF,  p-value: < 0.00000000000000022
##
##
## Value of test-statistic is: -13.4492
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau1 -2.58 -1.95 -1.62
```

The Verdict:
We reject null hypotesysi of non stationarity

VERIFICARE PRESENZA DI STAGIONALITA

```r
# Calcola lo spettro di potenza
?spec.pgram
spec_raw <- spec.pgram(as.numeric(train), spans = c(3,3), taper = 0.1, plot = TRUE,log='no')
```




```r
# Trova la frequenza con il picco massimo
max_freq <- spec_raw$freq[which.max(spec_raw$spec)]
periodo_dominante <- 1 / max_freq
cat(periodo_dominante)
```

```
## 2.045455
```

Il Periodogramma per l'Analisi Funzionamento: Il grafico del periodogramma mostra una sequenza di picchi. Ogni picco rappresenta la forza di una specifica frequenza nella serie. Identificazione della Stagionalità: Un picco pronunciato in corrispondenza di una determinata frequenza (specialmente frequenze basse, vicine a 0.1 per dati annuali) indica la presenza di una componente periodica o stagionale.
Un picco a una frequenza che indica un ciclo di 12 mesi segnala una forte stagionalità annuale

Ecco come leggere questo Smoothed Periodogram per capire cosa sta succedendo allo spread IT10DE10:

ANTOHER TEST TO DETETECT SEASONALITY (sarebbe da rifari sui residui del modello, perche cosi dice ben poco dato con non sappiamo qual e lag provoca il rifiuto)

```
#install.packages("seastests")
library(seastests)

# Creiamo l'oggetto TS (testiamo la frequenza 12 che vedevamo nell'ACF)
serie_ts <- ts(as.numeric(train), frequency = 12)

?isSeasonal
# Il test 'combined' ti dice se la stagionalità è presente
check <- isSeasonal(serie_ts, test = "combined")

if(check) {
  cat("La stagionalità a 12 è presente. Procedi con SARIMA.")
} else {
  cat("Nessuna stagionalità significativa trovata.")
}
```

```
## Nessuna stagionalità significativa trovata.
```

BEST MODEL WITH train <- ARMA(4,0,0) BEST MODEL WITH train2 <- ARMA(0,0,3)

TRY ALSO WITH seasonal=True to test stagionality significance at lag 8

```
test_freqs <- c(1,8, 12) # Le frequenze plausibili
risultati <- data.frame(Freq = test_freqs, AIC = NA)

for(i in 1:length(test_freqs)) {
  # Crea la TS con la frequenza del loop
  test_ts <- ts(as.numeric(train), frequency = test_freqs[i])

  # auto.arima testa se la stagionalità a quella frequenza aiuta il modello
  fit <- auto.arima(test_ts, max.q = 10, max.p = 10,seasonal = TRUE,method="ML",stepwise = FALSE,approxi
  risultati$AIC[i] <- fit$aic
}

# Trova la frequenza migliore
best_f <- risultati$Freq[which.min(risultati$AIC)]
cat("Il verdetto statistico è: usa la frequenza", best_f)
```

```
## Il verdetto statistico è: usa la frequenza 12
```

```
train_ts12 <- ts(as.numeric(train), frequency = 12)
m1_s12 <- auto.arima(train_ts12,max.q = 10, max.p = 10,method="ML",
                        seasonal = TRUE,
```

```r
                        stepwise = FALSE,
                        approximation = FALSE,
                        trace = FALSE)
summary(m1_s12)
```

```
## Series: train_ts12
## ARIMA(4,0,0)(0,0,1)[12] with zero mean
##
## Coefficients:
##           ar1    ar2      ar3      ar4    sma1
##       -0.0235  0.093  -0.1264  -0.1254  0.1108
## s.e.   0.0433  0.043   0.0429   0.0433  0.0429
##
## sigma^2 = 117.4:  log likelihood = -1985.84
## AIC=3983.69   AICc=3983.85   BIC=4009.25
##
## Training set error measures:
##                       ME     RMSE      MAE MPE MAPE      MASE         ACF1
## Training set -0.07453733 10.78145 7.562619 NaN  Inf 0.7079651 -0.004398738
```

BEST MODEL WITH train <- SARIMA(4,0,0)(1,0,0)[12] BEST MODEL WITH train2 <- ARIMA(0,0,3)

```r
m1 <- auto.arima(as.numeric(train),max.q = 10, max.p = 10,method="ML",
                        seasonal = FALSE,
                        stepwise = FALSE,
                        approximation = FALSE,
                        trace = FALSE)

summary(m1)
```

```
## Series: as.numeric(train)
## ARIMA(4,0,0) with zero mean
##
## Coefficients:
##           ar1     ar2      ar3      ar4
##       -0.0243  0.0925  -0.1275  -0.1203
## s.e.   0.0434  0.0430   0.0430   0.0433
##
## sigma^2 = 118.6:  log likelihood = -1989.08
## AIC=3988.16   AICc=3988.27   BIC=4009.46
##
## Training set error measures:
##                       ME     RMSE     MAE MPE MAPE      MASE         ACF1
## Training set -0.07995982 10.84989 7.59486 NaN  Inf 0.6983349 -0.003543211
```

BEST MODEL WITH train <- ARIMA(4,0,0) BEST MODEL WITH train2 <- ARIMA(0,0,3)

BISOGNA CONTINUARE A SVILUPPARE TEMA DELLA STAGIONALITA', MA NEL FRATTEMPO COSTRUIAMO LA PRIMA FORECAST SENZA ALCUNA STAGIONLITA' (USANDO QUINDI m1) PERCHE IL TEMPO STRINGE

```r
arma_assumptions <- function(m1_s12) {
  cat("\n==============================================================\n")
  cat("ARIMA model assumptions & Cycle Analysis:\n")

  # Estrazione corretta per modelli auto.arima/Arima
  # Usiamo i coefficienti AR per la stazionarietà
  ar_coefs <- m1$model$phi
  ma_coefs <- m1$model$theta

  # 1. Calcolo Radici
  ar_roots <- polyroot(c(1, -ar_coefs))
  ma_roots <- polyroot(c(1, ma_coefs))

  is_stationary <- all(abs(ar_roots) > 1)
  is_invertible <- all(abs(ma_roots) > 1)

  cat("Stationary:", is_stationary, "(Min mod:", round(min(abs(ar_roots)), 3), ")\n")
  cat("Invertible:", is_invertible, "(Min mod:", round(min(abs(ma_roots)), 3), ")\n")

  # 2. Analisi dei Business Cycles (Radici Complesse)
  # Cerchiamo se esistono radici con parte immaginaria significativa
  complex_roots <- ar_roots[Im(ar_roots) != 0]

  cat("\n--- Business Cycle Analysis ---\n")
  if(length(complex_roots) > 0) {
    cat("The model IMPLIES the existence of cycles.\n")
    cat("Reason: The AR characteristic equation has complex conjugate roots.\n")

    # Calcolo del periodo del ciclo (se ti serve per l'analisi)
    # Periodo = 2*pi / acos(Re(root)/abs(root))
    root <- complex_roots[1]
    period <- 2 * pi / atan2(Im(root), Re(root))
    cat("Estimated average cycle period:", round(abs(period), 2), "units of time.\n")
  } else {
    cat("The model DOES NOT imply cycles.\n")
    cat("Reason: All AR roots are real; the model returns to equilibrium monotonically.\n")
  }

  # 3. Check for common roots
  common_roots <- FALSE
  if(length(ar_roots) > 0 && length(ma_roots) > 0) {
    for(ar in ar_roots) {
      for(ma in ma_roots) {
        if(abs(ar - ma) < 0.05) common_roots <- TRUE
      }
    }
  }

  cat("\n--- Root Validation ---\n")
  if(common_roots) {
    cat("Common roots detected, assumption violation\n")
  } else {
    cat("No common roots detected, assumption validated\n")
```

```r
  }
  cat("==============================================================\n")
}

# Esecuzione
arma_assumptions(m1_s8)
```

```
##
## ==============================================================
## ARIMA model assumptions & Cycle Analysis:
## Stationary: TRUE (Min mod: 1.544 )

## Warning in min(abs(ma_roots)): no non-missing arguments to min; returning Inf

## Invertible: TRUE (Min mod: Inf )
##
## --- Business Cycle Analysis ---
## The model IMPLIES the existence of cycles.
## Reason: The AR characteristic equation has complex conjugate roots.
## Estimated average cycle period: 7.64 units of time.
##
## --- Root Validation ---
## No common roots detected, assumption validated
## ==============================================================
```

```r
cat("\n==============================================================\n")
```

Residual's check

```
##
## ==============================================================
```

```r
cat("\nARMA residuals check:\n")
```

```
##
## ARMA residuals check:
```

```r
# ARIMA residuals
arima_res <- residuals(m1_s12)

arma_residual <- function(res) {
  cat("\n==============================================================\n")
  cat("ANALISI APPROFONDITA DEI RESIDUI\n")
  cat("==============================================================\n")

# 1. Ljung-Box (Media e Autocorrelazione)
  lb_arima <- Box.test(res, lag = 10, type = "Ljung-Box")
  cat("\n [1]LB test on ARIMA residuals (10 lags) p-value: ", round(lb_arima$p.value, 4), "\n")
```

```r
  if(lb_arima$p.value >= 0.05) cat("ARIMA model cleared the mean\n") else cat("ARIMA model DID NOT clea
#Regola di Hyndman (autore di forecast): Per dati non stagionali si usa $h = 10$, per dati stagionali s

# 2. Durbin-Watson (Autocorrelazione al lag 1)
# Nota: Il DW classico si usa su modelli lineari, sui residui ARIMA è indicativo
  dw_test <- dwtest(as.vector(res) ~ 1)
  cat("\n[2] Durbin-Watson Test p-value: ", round(dw_test$p.value, 4))
  if(dw_test$p.value >= 0.05) cat(" -> OK: No autocorrelazione lag 1\n") else cat(" -> KO: Autocorrelaz

# 3. Breusch-Godfrey Test (Autocorrelazione di ordine superiore)
  bg_test <- bgtest(as.vector(res) ~ 1, order = 8)
  cat("\n[3] Breusch-Godfrey Test (order 8) p-value: ", round(bg_test$p.value, 4))
  if(bg_test$p.value >= 0.05) cat(" -> OK: No autocorrelazione superiore\n") else cat(" -> KO: Autocorre

# 4. Jarque-Bera Test (Normalità dei residui)
  jb_test <- jarque.bera.test(res)
  cat("\n[4] Jarque-Bera Test p-value: ", round(jb_test$p.value, 4))
  if(jb_test$p.value >= 0.05) cat(" -> OK: Residui Normali\n") else cat(" -> KO: Residui NON Normali (co

  # 5. Effetti ARCH (Eteroschedasticità)
  lb_arima_2 <- Box.test(res^2, lag = 10, type = "Ljung-Box")
  cat("\n [5]LB test on ARIMA squared residuals (10 lags) p-value: ", round(lb_arima_2$p.value, 4), "\n
  if(lb_arima_2$p.value < 0.05) cat("ARIMA model has residual variance to explain. GARCH model needed\n

  # ARCH test
  arch_test_res <- ArchTest(res, lags = 5)
  cat(sprintf("\n [5]ARCH Test on ARIMA (5 lags) residuals:\n LM Test - statistic: %.4f, p-value: %.4f\n
              arch_test_res$statistic, arch_test_res$p.value))
  if(arch_test_res$p.value < 0.05) cat("ARCH effects confirmed. GARCH modelling needed\n") else cat("No
}

print(arma_residual(arima_res))
```

```
##
## ============================================================
## ANALISI APPROFONDITA DEI RESIDUI
## ============================================================
##
##  [1]LB test on ARIMA residuals (10 lags) p-value:  0.9816
## ARIMA model cleared the mean
##
## [2] Durbin-Watson Test p-value:  0.5371 -> OK: No autocorrelazione lag 1
##
## [3] Breusch-Godfrey Test (order 8) p-value:  0.9841 -> OK: No autocorrelazione superiore
##
## [4] Jarque-Bera Test p-value:  0 -> KO: Residui NON Normali (code grasse)
##
##  [5]LB test on ARIMA squared residuals (10 lags) p-value:  0
## ARIMA model has residual variance to explain. GARCH model needed
##
##  [5]ARCH Test on ARIMA (5 lags) residuals:
##  LM Test - statistic: 95.5222, p-value: 0.0000
## ARCH effects confirmed. GARCH modelling needed
```

```
## NULL
```

Are all the coefficients significant?

```r
check_coef_significance <- function(model) {
  # Estraiamo coefficienti e standard errors
  coefs <- model$coef
  se <- sqrt(diag(model$var.coef))

  # Calcoliamo la t-statistica (z-score)
  t_stats <- coefs / se

  # Calcoliamo i p-value (distribuzione normale a due code)
  p_values <- (1 - pnorm(abs(t_stats))) * 2

  # Creiamo una tabella riassuntiva
  results <- data.frame(
    Coefficiente = round(coefs, 4),
    Std_Error = round(se, 4),
    t_stat = round(t_stats, 4),
    p_value = round(p_values, 4),
    Significativo_95 = ifelse(p_values < 0.05, "SI", "NO")
  )

  cat("\nAnalisi di Significatività dei Coefficienti:\n")
  print(results)

  cat("\nNota: Un coefficiente è significativo al 95% se |t_stat| > 1.96\n")
  print(results)
}
check_coef_significance(m1_s12)
```

```
##
## Analisi di Significatività dei Coefficienti:
##      Coefficiente Std_Error  t_stat p_value Significativo_95
## ar1       -0.0235    0.0433 -0.5411  0.5885               NO
## ar2        0.0930    0.0430  2.1620  0.0306               SI
## ar3       -0.1264    0.0429 -2.9442  0.0032               SI
## ar4       -0.1254    0.0433 -2.8938  0.0038               SI
## sma1       0.1108    0.0429  2.5844  0.0098               SI
##
## Nota: Un coefficiente è significativo al 95% se |t_stat| > 1.96
##      Coefficiente Std_Error  t_stat p_value Significativo_95
## ar1       -0.0235    0.0433 -0.5411  0.5885               NO
## ar2        0.0930    0.0430  2.1620  0.0306               SI
## ar3       -0.1264    0.0429 -2.9442  0.0032               SI
## ar4       -0.1254    0.0433 -2.8938  0.0038               SI
## sma1       0.1108    0.0429  2.5844  0.0098               SI
```

ARE ALL THE COEFFICIENTS OF m1 STATISTICALLY SIGNIFICANT? t ratios are given by estimated coefficients divided by their standard errors; the latter are given by the square root of the element on the main diagonal (variance of coef)

```r
names(m1_s12)
```

```
##  [1] "coef"      "sigma2"    "var.coef"  "mask"      "loglik"    "aic"
##  [7] "arma"      "residuals" "call"      "series"    "code"      "n.cond"
## [13] "nobs"      "model"     "bic"       "aicc"      "x"         "fitted"
```

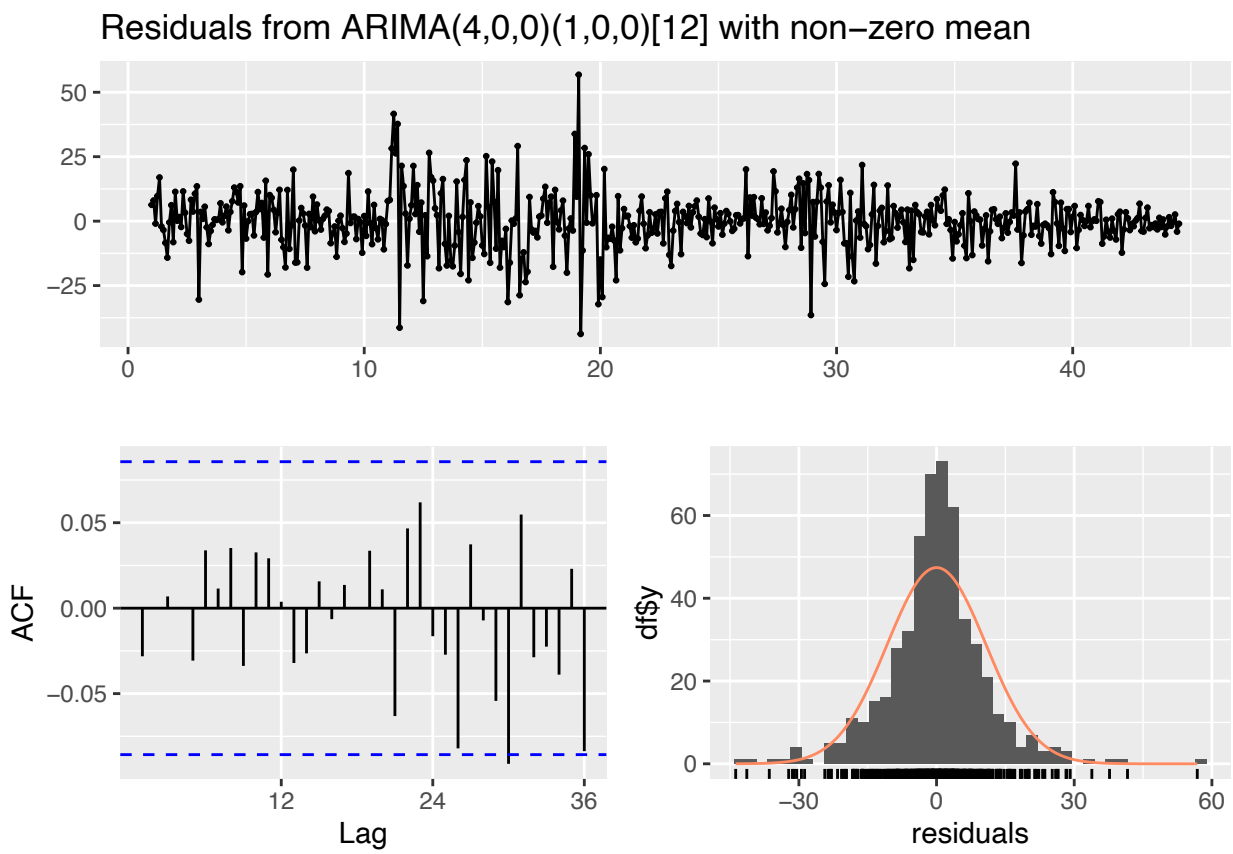
```r
summary(m1_s12)
```

```
## Series: train_ts12
## ARIMA(4,0,0)(0,0,1)[12] with zero mean
##
## Coefficients:
##           ar1    ar2      ar3      ar4    sma1
##       -0.0235  0.093  -0.1264  -0.1254  0.1108
## s.e.   0.0433  0.043   0.0429   0.0433  0.0429
##
## sigma^2 = 117.4:  log likelihood = -1985.84
## AIC=3983.69   AICc=3983.85   BIC=4009.25
##
## Training set error measures:
##                       ME     RMSE      MAE MPE MAPE      MASE         ACF1
## Training set -0.07453733 10.78145 7.562619 NaN  Inf 0.7079651 -0.004398738
```

```r
t.ratios = m1_s12$coef/sqrt(diag(m1_s12$var.coef))
t.ratios
```

```
##        ar1        ar2        ar3        ar4       sma1
## -0.5410783  2.1620023 -2.9441572 -2.8937682  2.5844377
```

```r
# Il vettore fixed deve avere la stessa lunghezza del numero di parametri (3 MA + 1 media)
# Usiamo NA per i parametri da stimare e 0 per quelli da bloccare
m1_s12 <- Arima(train_ts12,
                order = c(4,0,0),
                seasonal = list(order = c(1,0,0), period = 12),
                fixed = c(0, NA, NA, NA, NA, NA)) # 4 AR + 1 SAR + 1 mean
checkresiduals(m1_s12)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(4,0,0)(1,0,0)[12] with non-zero mean
## Q* = 11.416, df = 19, p-value = 0.9091
##
## Model df: 5.   Total lags used: 24
```

ORA FACCIAMO STA BENEDETTA PREVISIONE:

IN SAMPLE PREDICTIONS

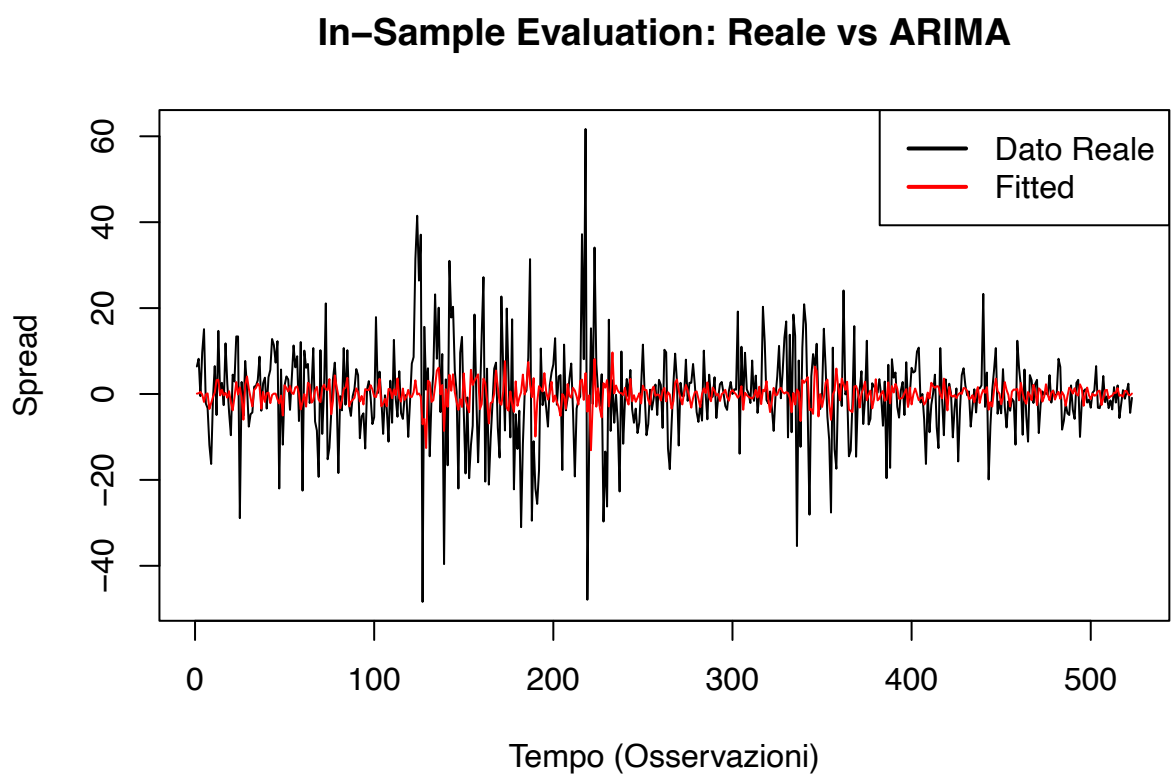
```
# Estraiamo i valori fittati dal modello m1
fitted_values <- fitted(m1_s12)
length(fitted_values)
```

```
## [1] 523
```

```
length(train)
```

```
## [1] 523
```

```r
# Visualizzazione grafica: Reale vs Fittato
plot(as.numeric(train), type="l", col="black", lwd=1,
     main="In-Sample Evaluation: Reale vs ARIMA",
     ylab="Spread", xlab="Tempo (Osservazioni)")
lines(as.numeric(na.omit(fitted_values)), col="red")
legend("topright", legend=c("Dato Reale", "Fitted"), col=c("black", "red"), lty=1, lwd=2)
```



IN SAMPLE EVALUATION

```r
residuals <- train - fitted_values
```

```
## Warning: Incompatible methods ("Ops.zoo", "Ops.ts") for "-"
```

```r
# Core metrics
accuracy(as.numeric(train), as.numeric(fitted_values))
```

```
##                     ME     RMSE      MAE      MPE     MAPE
## Test set -0.004486872 10.78572 7.574807 1154.645 3344.502
```

```r
# R-squared (will be 0 or near 0 - expected for mean model)
SS_res <- sum(residuals^2, na.rm = TRUE)
SS_tot <- sum((train - mean(train, na.rm = TRUE))^2, na.rm = TRUE)
R2 <- 1 - SS_res / SS_tot
cat("R2 del modello:", round(R2, 2), "%")
```

```
## R2 del modello: 0.05 %
```

```r
# Calcoliamo i segni (1 per positivo, -1 per negativo)
sign_actual <- sign(train)
sign_fitted <- sign(fitted_values)

# Creiamo un vettore logico: TRUE se i segni sono uguali
hits <- (sign_actual == sign_fitted)
```

```
## Warning: Incompatible methods ("Ops.zoo", "Ops.ts") for "=="
```

```r
# Calcoliamo la percentuale di successo (escludendo eventuali NA)
DA <- mean(hits, na.rm = TRUE) * 100
DA
```

```
## [1] 51.62524
```

OUT-SAMPLE PREDICTION AND EVALUATION

```r
# 1. Applichiamo il modello alla serie completa (genera tutti i forecast 1-step)
refit <- Arima(diff_spread, model = m1_s12)

# 2. Estraiamo solo i valori fittati che corrispondono al periodo di TEST
# (Supponiamo che il tuo test set sia lungo 33 osservazioni)
pred_out <- tail(fitted(refit), n_test)
reali_out <- tail(as.numeric(diff_spread), n_test)

# 3. Calcolo metriche Out-of-Sample
accuracy(pred_out, reali_out)
```

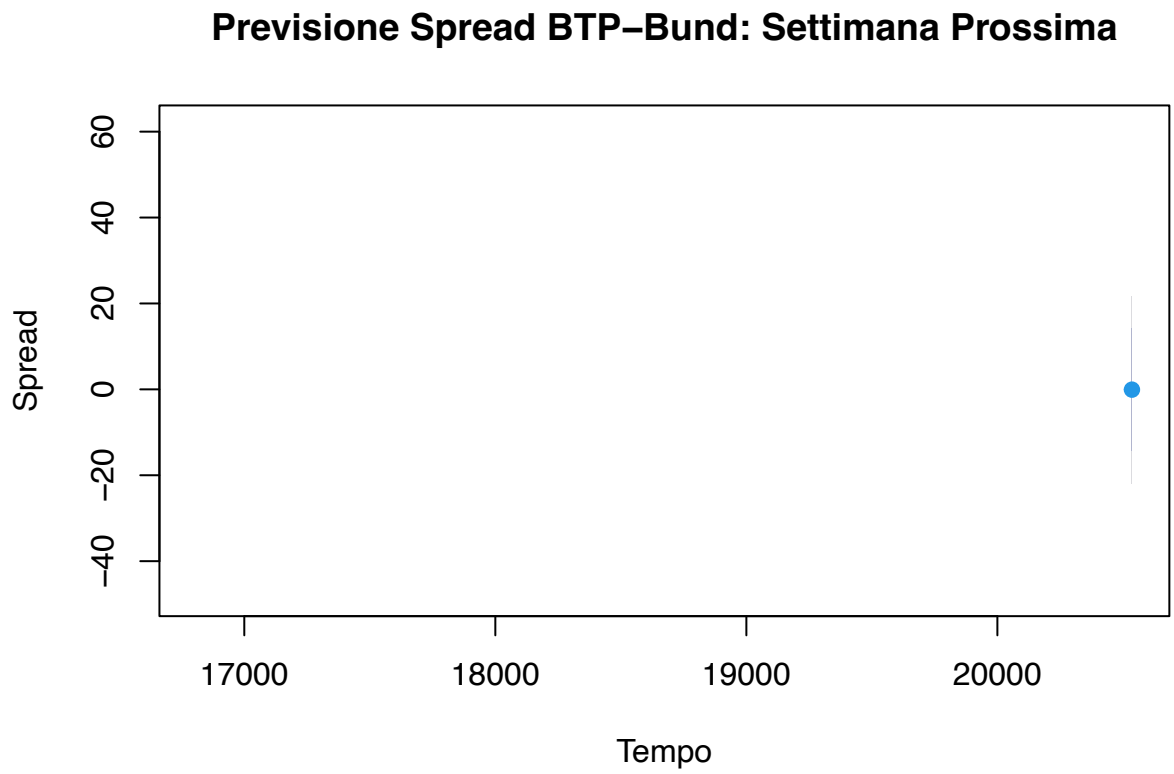
```
##                ME     RMSE      MAE      MPE     MAPE
## Test set 1.136854 1.531966 1.136854 56.07518 56.07518
```

FORECAST DI SETTIMANA PROSSIMA (diff_spreads)

```r
library(forecast)
# 2. Generiamo la previsione per la prossima settimana
forecast_next_week <- forecast(refit, h = 1)

# 3. Visualizziamo il risultato
print(forecast_next_week)
```

```
##       Point Forecast     Lo 80    Hi 80     Lo 95    Hi 95
## 20536    -0.06696146 -14.30904 14.17511 -21.84834 21.71441
```

```r
# 4. Grafico della previsione con intervalli di confidenza
plot(forecast_next_week, main = "Previsione Spread BTP-Bund: Settimana Prossima",
     ylab = "Spread", xlab = "Tempo")
```



Lo 80 / Hi 80 (da -21.79 a +22.18): Il modello è sicuro all'80% che la variazione dello spread oscillerà tra una discesa di 21.8 punti e una salita di 22.2 punti. È un range abbastanza ampio che riflette la volatilità tipica dei mercati.

Lo 95 / Hi 95 (da -33.43 a +33.82): Il modello è sicuro al 95% (standard statistico per la tesi) che la variazione non supererà i ±33 punti base.

Nota tecnica: Se la prossima settimana lo spread dovesse muoversi di 50 punti, sapresti che è successo un evento "fuori dal comune" (un outlier) che il modello non poteva prevedere. TRASFORMAZIONE IN spread:

```
# 1. Recuperiamo l'ultimo valore REALE dello spread (non differenziato)
ultimo_spread_reale <- tail(spread, 1)

# 2. Otteniamo la previsione della differenza (il "delta")
previsione_diff <- forecast(m1_s12, h = 1)$mean[1]

# 3. Calcoliamo lo spread atteso
spread_previsto <- ultimo_spread_reale + previsione_diff

cat("Lo spread previsto per settimana prossima è:", spread_previsto)
```

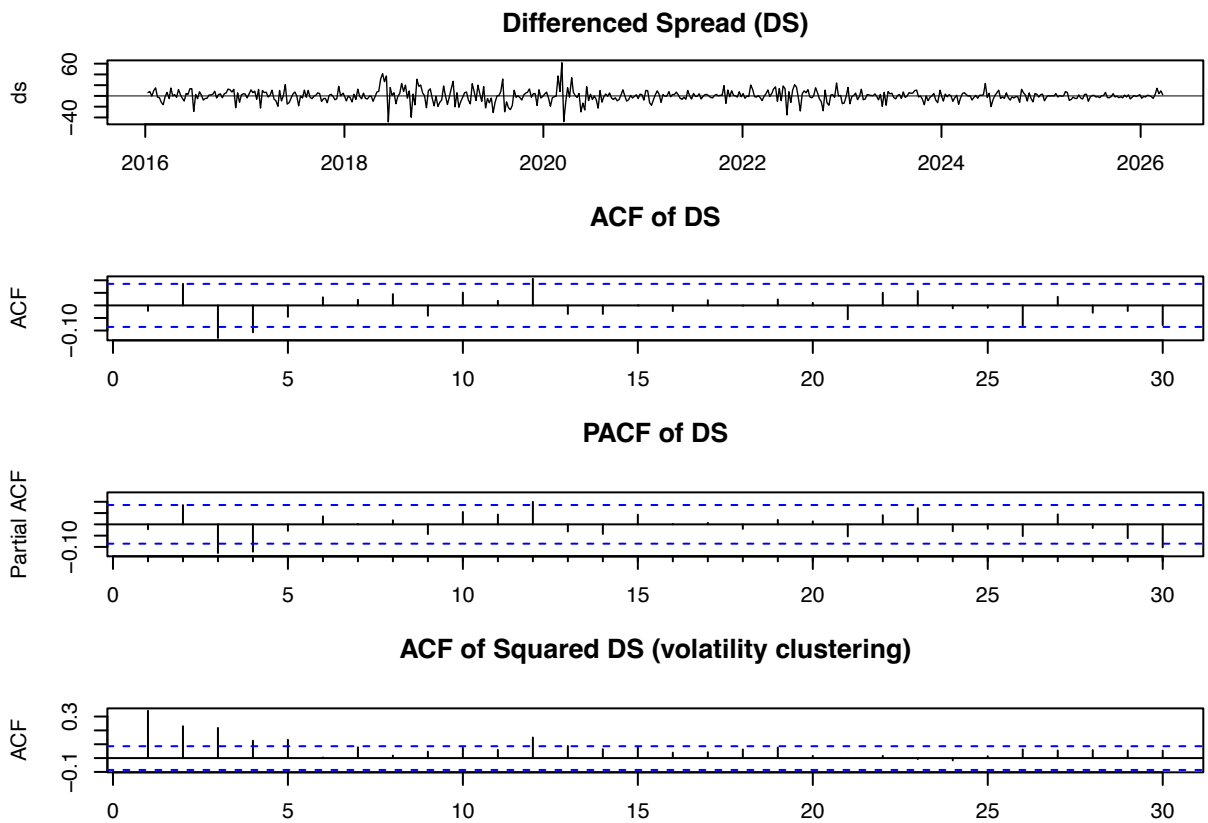
```
## Lo spread previsto per settimana prossima è: 91.96137
```

CONSIDERARE DI FARE UNA CROSS VALIDATION PER UNA ROBA PIU ACCURATA

Volatility Analysis

Before fitting ARMA, we check for autocorrelation and ARCH effects on `diff_spread` (= `train` + `test`).

```
par(mfrow = c(4, 1), mar = c(3, 4, 3, 2))

plot(index(as.xts(diff_spread)), coredata(as.xts(diff_spread)), type = "l", lwd = 0.7,
     main = "Differenced Spread (DS)", xlab = "", ylab = "ds")
abline(h = 0, col = "black", lwd = 0.5)

Acf(as.numeric(train),   lag.max = 30, main = "ACF of DS")
Pacf(as.numeric(train),  lag.max = 30, main = "PACF of DS")
Acf(as.numeric(train)^2, lag.max = 30, main = "ACF of Squared DS (volatility clustering)")
```




```
par(mfrow = c(1, 1))
```

Check for autocorrelation and ARCH effects on `train`:

```
# Ljung-Box on DS
lb_ds <- Box.test(as.numeric(train), lag = 10, type = "Ljung-Box")
cat(sprintf("LB Test on DS p-value: %.4f\n", lb_ds$p.value))
```

```
## LB Test on DS p-value: 0.0068
```

```r
if (lb_ds$p.value < 0.05) cat("Autocorrelation detected - mean modelling needed\n") else cat("DS are Whi
```

```
## Autocorrelation detected - mean modelling needed
```

```r
# Ljung-Box on DS^2
lb_ds2 <- Box.test(as.numeric(train)^2, lag = 10, type = "Ljung-Box")
cat(sprintf("LB Test on DS^2 p-value: %.4f\n", lb_ds2$p.value))
```

```
## LB Test on DS^2 p-value: 0.0000
```

```r
if (lb_ds2$p.value < 0.05) cat("Volatility clustering confirmed\n") else cat("No volatility clustering c
```

```
## Volatility clustering confirmed
```

```r
# ARCH test
arch_test <- ArchTest(as.numeric(train), lags = 5)
cat(sprintf("ARCH Test (5 lags) p-value: %.4f\n", arch_test$p.value))
```

```
## ARCH Test (5 lags) p-value: 0.0000
```

```r
if (arch_test$p.value < 0.05) cat("ARCH effects confirmed - GARCH modelling needed\n") else cat("No ARCH
```

```
## ARCH effects confirmed - GARCH modelling needed
```

Model 2: ARMA

We use `auto.arima` to select the best ARMA(p,q) order by AIC/BIC on the training set.

```r
arima_model <- auto.arima(as.numeric(train),
  start.p = 1, start.q = 1,
  max.p = 5, max.q = 5,
  seasonal = FALSE,
  stepwise = FALSE, trace = FALSE, method = "ML")

print(summary(arima_model))
```

```
## Series: as.numeric(train)
## ARIMA(4,0,0) with zero mean
##
## Coefficients:
##           ar1     ar2      ar3      ar4
##       -0.0243  0.0925  -0.1275  -0.1203
## s.e.   0.0434  0.0430   0.0430   0.0433
##
## sigma^2 = 118.6:  log likelihood = -1989.08
## AIC=3988.16   AICc=3988.27   BIC=4009.46
##
## Training set error measures:
##                       ME     RMSE     MAE MPE MAPE      MASE         ACF1
## Training set -0.07995982 10.84989 7.59486 NaN  Inf 0.6983349 -0.003543211
```

Check ARMA assumptions (stationarity, invertibility, no common roots):

```
arma_assumptions <- function(model) {
  ar_roots <- polyroot(c(1, -model$model$phi))
  ma_roots <- polyroot(c(1, model$model$theta))

  is_stationary <- all(abs(ar_roots) > 1)
  is_invertible  <- all(abs(ma_roots) > 1)

  cat("Stationary:", is_stationary, "\n")
  cat("Invertible:", is_invertible,  "\n")

  common_roots <- FALSE
  if (length(ar_roots) > 0 && length(ma_roots) > 0) {
    for (ar in ar_roots) {
      for (ma in ma_roots) {
        if (isTRUE(all.equal(ar, ma, tolerance = 1e-3))) common_roots <- TRUE
      }
    }
  }
  if (common_roots) cat("Common roots detected - assumption violation\n") else cat("No common roots - as
}

arma_assumptions(arima_model)
```

```
## Stationary: TRUE
## Invertible: TRUE
## No common roots - assumption validated
```

ARMA Residual Check

```
arima_res <- residuals(arima_model)

lb_res  <- Box.test(arima_res,   lag = 10, type = "Ljung-Box")
lb_res2 <- Box.test(arima_res^2, lag = 10, type = "Ljung-Box")
arch_res <- ArchTest(arima_res, lags = 5)

cat(sprintf("LB on ARIMA residuals p-value:   %.4f\n", lb_res$p.value))
```

```
## LB on ARIMA residuals p-value:   0.9704
```

```
if (lb_res$p.value >= 0.05) cat("ARIMA cleared the mean\n") else cat("ARIMA DID NOT clear the mean\n")
```

```
## ARIMA cleared the mean
```

```
cat(sprintf("LB on ARIMA residuals^2 p-value: %.4f\n", lb_res2$p.value))
```

```
## LB on ARIMA residuals^2 p-value: 0.0000
```

```r
if (lb_res2$p.value < 0.05) cat("Residual variance remains - GARCH needed\n") else cat("Residual varian
```

```
## Residual variance remains - GARCH needed
```

```r
cat(sprintf("ARCH Test on residuals p-value:  %.4f\n", arch_res$p.value))
```

```
## ARCH Test on residuals p-value:  0.0000
```

```r
if (arch_res$p.value < 0.05) cat("ARCH effects confirmed - GARCH modelling needed\n") else cat("No ARCH
```

```
## ARCH effects confirmed - GARCH modelling needed
```

Model 3: ARMA-GARCH

Grid search over ARMA(p,q)-GARCH(p,q) combinations using parallelisation. Best model selected by BIC.

```r
library(parallel)
library(R.utils)

best_garch <- function(y,
                       ar_lags = 0:3, ma_lags = 0:3,
                       garch_p = 1:2,  garch_q = 1:2,
                       models  = c("sGARCH", "eGARCH"),
                       dists   = c("norm", "std", "sstd"),
                       criteria = "bic", timeout = 20) {

  combos <- expand.grid(
    ar_lag = ar_lags, ma_lag = ma_lags,
    garch_p = garch_p, garch_q = garch_q,
    model = models, dist = dists,
    stringsAsFactors = FALSE
  )

  cat("Total combinations to fit:", nrow(combos), "\n")
  cat("Running on", detectCores() - 1, "cores\n")
  cat("It could take a couple of minutes...\n\n")

  fit_one <- function(i) {
    arp <- combos$ar_lag[i]; map <- combos$ma_lag[i]
    gp  <- combos$garch_p[i]; gq <- combos$garch_q[i]
    model <- combos$model[i]; dist <- combos$dist[i]

    spec <- ugarchspec(
      variance.model    = list(model = model, garchOrder = c(gp, gq)),
      mean.model        = list(armaOrder = c(arp, map), include.mean = TRUE),
      distribution.model = dist
    )

    fit <- tryCatch(
      withTimeout(
```

```r
        ugarchfit(spec = spec, data = y, solver = "hybrid", solver.control = list(trace = 0)),
        timeout = timeout, onTimeout = "silent"
      ),
      error = function(e) NULL, warning = function(w) NULL
    )

    if (!is.null(fit) && inherits(fit, "uGARCHfit")) {
      ic <- infocriteria(fit)
      return(data.frame(
        ar_p = arp, ma_q = map, garch_p = gp, garch_q = gq,
        model = model, dist = dist,
        aic = as.numeric(round(ic["Akaike", ] * length(y), 2)),
        bic = as.numeric(round(ic["Bayes",  ] * length(y), 2)),
        stringsAsFactors = FALSE
      ))
    }
    return(NULL)
  }

  results_list <- mclapply(seq_len(nrow(combos)), fit_one, mc.cores = detectCores() - 1)
  successful   <- Filter(Negate(is.null), results_list)

  cat("Successful fits:", length(successful), "/", nrow(combos), "\n")
  results_df <- bind_rows(successful) |> arrange(.data[[criteria]])
  return(results_df)
}

results <- best_garch(as.numeric(train))
```

```
## Total combinations to fit: 384
## Running on 9 cores
## It could take a couple of minutes...
##
## Successful fits: 384 / 384
```

```r
cat("\nTop models by BIC:\n")
```

```
##
## Top models by BIC:
```

```r
print(head(results))
```

```
##   ar_p ma_q garch_p garch_q  model dist     aic     bic
## 1    0    0       1       1 eGARCH  std 3790.92 3816.48
## 2    0    0       1       2 eGARCH  std 3792.18 3822.00
## 3    0    0       1       1 eGARCH sstd 3792.80 3822.62
## 4    1    0       1       1 eGARCH  std 3792.91 3822.73
## 5    0    1       1       1 eGARCH  std 3792.91 3822.73
## 6    0    0       2       1 eGARCH  std 3791.47 3825.54
```

Fit the best model:

```r
arma_best_order  <- c(results$ar_p[1], results$ma_q[1])
garch_best_order <- c(results$garch_p[1], results$garch_q[1])
best_dist        <- results$dist[1]
best_mod         <- results$model[1]

cat("Best ARMA order:", arma_best_order, "\n")
```

```
## Best ARMA order: 0 0
```

```r
cat("Best GARCH order:", garch_best_order, "\n")
```

```
## Best GARCH order: 1 1
```

```r
garch_spec <- ugarchspec(
  variance.model   = list(model = best_mod, garchOrder = garch_best_order),
  mean.model       = list(armaOrder = arma_best_order, include.mean = TRUE),
  distribution.model = best_dist
)

garch_fit <- ugarchfit(spec = garch_spec, data = as.numeric(train), solver = "hybrid")
show(garch_fit)
```

```
##
## *---------------------------------*
## *          GARCH Model Fit        *
## *---------------------------------*
##
## Conditional Variance Dynamics
## -----------------------------------
## GARCH Model  : eGARCH(1,1)
## Mean Model   : ARFIMA(0,0,0)
## Distribution : std
##
## Optimal Parameters
## ------------------------------------
##         Estimate  Std. Error     t value Pr(>|t|)
## mu     -0.103416    0.243711    -0.42434 0.671318
## omega   0.034467    0.006583     5.23588 0.000000
## alpha1  0.181055    0.032081     5.64364 0.000000
## beta1   0.990425    0.000066 14928.85594 0.000000
## gamma1  0.116172    0.016110     7.21123 0.000000
## shape   4.713946    0.978408     4.81797 0.000001
##
## Robust Standard Errors:
##         Estimate  Std. Error     t value Pr(>|t|)
## mu     -0.103416    0.196842    -0.52538 0.599322
## omega   0.034467    0.007112     4.84659 0.000001
## alpha1  0.181055    0.033962     5.33117 0.000000
## beta1   0.990425    0.000085 11638.08068 0.000000
## gamma1  0.116172    0.015991     7.26499 0.000000
## shape   4.713946    1.103118     4.27329 0.000019
##
```

```
## LogLikelihood : -1889.459
##
## Information Criteria
## ------------------------------------
##
## Akaike       7.2484
## Bayes        7.2973
## Shibata      7.2481
## Hannan-Quinn 7.2675
##
## Weighted Ljung-Box Test on Standardized Residuals
## ------------------------------------
##                         statistic p-value
## Lag[1]                     0.1569 0.69199
## Lag[2*(p+q)+(p+q)-1][2]    0.9242 0.52349
## Lag[4*(p+q)+(p+q)-1][5]    6.1741 0.08183
## d.o.f=0
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## ------------------------------------
##                         statistic p-value
## Lag[1]                      2.707 0.09990
## Lag[2*(p+q)+(p+q)-1][5]     9.718 0.01087
## Lag[4*(p+q)+(p+q)-1][9]    12.304 0.01546
## d.o.f=2
##
## Weighted ARCH LM Tests
## ------------------------------------
##             Statistic Shape Scale P-Value
## ARCH Lag[3]     1.326 0.500 2.000  0.2496
## ARCH Lag[5]     2.266 1.440 1.667  0.4153
## ARCH Lag[7]     2.835 2.315 1.543  0.5453
##
## Nyblom stability test
## ------------------------------------
## Joint Statistic:  0.9551
## Individual Statistics:
## mu     0.09728
## omega  0.07430
## alpha1 0.03253
## beta1  0.06939
## gamma1 0.18115
## shape  0.03814
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:          1.49 1.68 2.12
## Individual Statistic:     0.35 0.47 0.75
##
## Sign Bias Test
## ------------------------------------
##                    t-value   prob sig
## Sign Bias          0.07704 0.9386
## Negative Sign Bias 0.39396 0.6938
```

```
## Positive Sign Bias 1.01560 0.3103
## Joint Effect       2.26520 0.5192
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## ------------------------------------
##   group statistic p-value(g-1)
## 1    20     9.658       0.9609
## 2    30    25.776       0.6374
## 3    40    33.405       0.7224
## 4    50    47.268       0.5436
##
##
## Elapsed time : 0.1466811
```

GARCH Residual Check

After removing both mean (ARMA) and variance (GARCH) effects, standardised residuals should be iid.

```r
std_res <- as.numeric(residuals(garch_fit, standardize = TRUE))

lb_garch  <- Box.test(std_res,   lag = 10, type = "Ljung-Box")
lb_garch2 <- Box.test(std_res^2, lag = 10, type = "Ljung-Box")
arch_garch <- ArchTest(std_res, lags = 5)

cat(sprintf("LB on std resid p-value:    %.4f (want > 0.05)\n", lb_garch$p.value))
```

```
## LB on std resid p-value:    0.0679 (want > 0.05)
```

```r
cat(sprintf("LB on std resid^2 p-value:  %.4f (want > 0.05)\n", lb_garch2$p.value))
```

```
## LB on std resid^2 p-value:  0.0380 (want > 0.05)
```

```r
cat(sprintf("ARCH LM on std resid p-value: %.4f (want > 0.05)\n", arch_garch$p.value))
```

```
## ARCH LM on std resid p-value: 0.0431 (want > 0.05)
```

Model Validation (Rolling Forecast)

1-step-ahead rolling forecast on the test set, refitting every week on a moving window.

```r
roll <- ugarchroll(
  spec            = garch_spec,
  data            = as.numeric(diff_spread),
  n.ahead         = 1,
  forecast.length = n_test,
  refit.every     = 1,
  refit.window    = "moving",
  window.size     = 300,
  solver          = "hybrid",
```

```r
  calculate.VaR   = TRUE,
  VaR.alpha       = c(0.05, 0.25, 0.75, 0.95)
)

roll_df <- as.data.frame(roll)
quants  <- as.data.frame(roll, which = "VaR")

forecast_df <- data.frame(
  pred_mu  = roll_df$Mu,
  pred_var = roll_df$Sigma^2,
  pred_vol = roll_df$Sigma,
  ci_q05   = quants[, "alpha(5%)"],
  ci_q25   = quants[, "alpha(25%)"],
  ci_q75   = quants[, "alpha(75%)"],
  ci_q95   = quants[, "alpha(95%)"],
  actual   = roll_df$Realized
)
rownames(forecast_df) <- rownames(roll_df)

print(tail(round(forecast_df, 4)))
```

```
##            pred_mu pred_var pred_vol   ci_q05  ci_q25 ci_q75  ci_q95 actual
## 1971-06-13 -0.4255   5.7552   2.3990  -4.1772 -1.7819 0.9308  3.3261   -4.6
## 1971-06-14 -0.5136   5.8375   2.4161  -4.3008 -1.8871 0.8599  3.2736    2.6
## 1971-06-15 -0.4188   7.2365   2.6901  -4.6778 -1.9862 1.1486  3.8403   14.3
## 1971-06-16 -0.3483  26.8546   5.1821  -8.4328 -3.2619 2.5653  7.7362    4.1
## 1971-06-17 -0.2506  25.6376   5.0634  -8.2474 -3.1824 2.6811  7.7462    9.4
## 1971-06-18 -0.3169  47.4979   6.8919 -11.0970 -4.2153 3.5814 10.4631    2.4
```

Mean Evaluation Metrics

```r
errors  <- forecast_df$actual - forecast_df$pred_mu
mae     <- mean(abs(errors))
mse     <- mean(errors^2)
rmse    <- sqrt(mse)
mape    <- mean(abs(errors / forecast_df$actual))
dir_acc <- mean(sign(forecast_df$actual) == sign(forecast_df$pred_mu))

cat(sprintf("MAE: %.4f | MSE: %.4f | RMSE: %.4f | MAPE: %.4f | Dir. Acc: %.2f%%\n",
            mae, mse, rmse, mape, dir_acc * 100))
```

```
## MAE: 4.3989 | MSE: 37.1476 | RMSE: 6.0949 | MAPE: 1.1137 | Dir. Acc: 30.00%
```

Volatility Evaluation Metrics

```r
proxy_var <- forecast_df$pred_mu^2
var_mae   <- mean(abs(proxy_var - forecast_df$pred_var))
var_mse   <- mean((proxy_var - forecast_df$pred_var)^2)
qlike     <- mean(log(forecast_df$pred_var) + proxy_var / forecast_df$pred_var)
```

```r
cat(sprintf("Vol MAE: %.4f | Vol MSE: %.4f | QLIKE: %.4f\n", var_mae, var_mse, qlike))
```

```
## Vol MAE: 14.4261 | Vol MSE: 390.1013 | QLIKE: 2.3755
```

Distribution Coverage Check

```r
coverage_90 <- mean(forecast_df$actual >= forecast_df$ci_q05 &
                    forecast_df$actual <= forecast_df$ci_q95)
coverage_50 <- mean(forecast_df$actual >= forecast_df$ci_q25 &
                    forecast_df$actual <= forecast_df$ci_q75)

cat(sprintf("90%% coverage: %.4f (target ~0.90)\n", coverage_90))
```

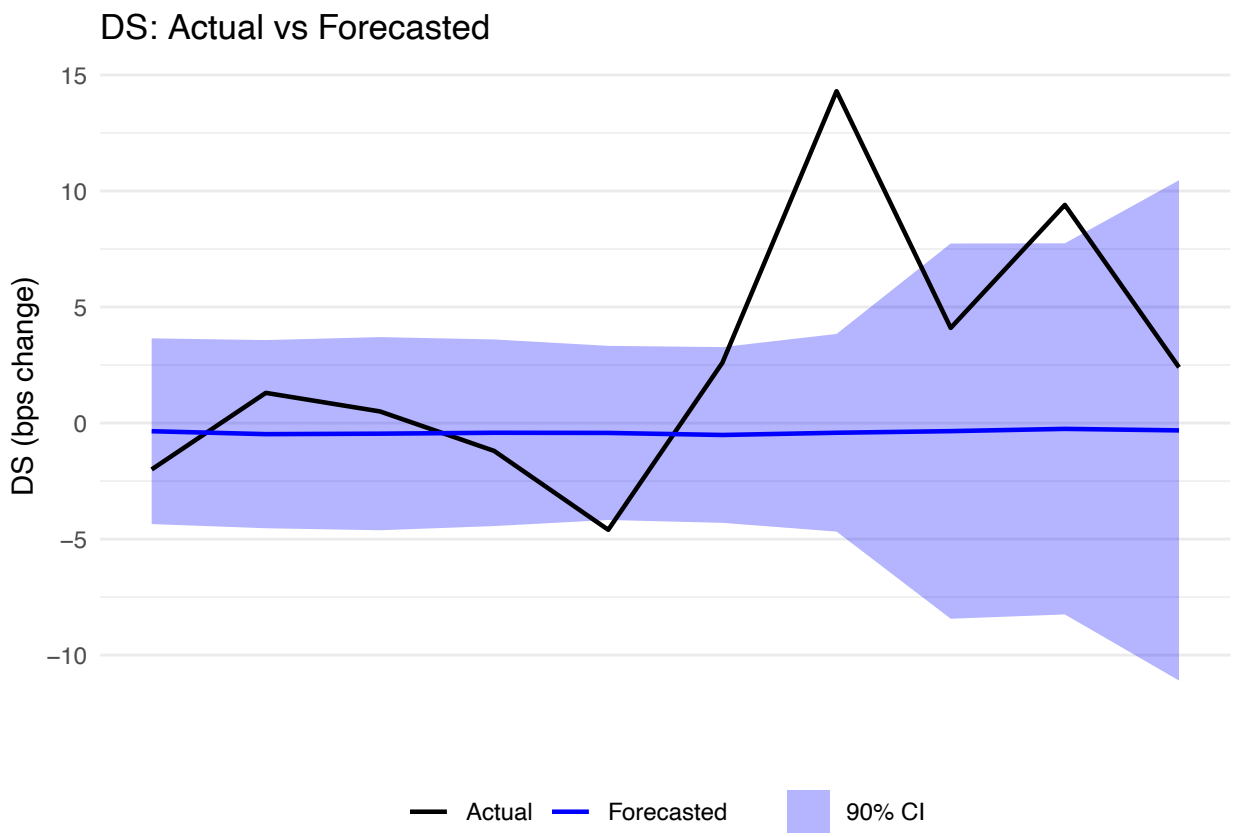
```
## 90% coverage: 0.7000 (target ~0.90)
```

```r
cat(sprintf("50%% coverage: %.4f (target ~0.50)\n", coverage_50))
```

```
## 50% coverage: 0.3000 (target ~0.50)
```

DS: Actual vs Forecasted

```r
forecast_df$date <- as.Date(rownames(forecast_df))

ggplot(forecast_df, aes(x = date)) +
  geom_ribbon(aes(ymin = ci_q05, ymax = ci_q95, fill = "90% CI"), alpha = 0.3) +
  geom_line(aes(y = actual,  color = "Actual"),    linewidth = 0.8) +
  geom_line(aes(y = pred_mu, color = "Forecasted"), linewidth = 0.8) +
  scale_color_manual(values = c("Actual" = "black", "Forecasted" = "blue")) +
  scale_fill_manual(values  = c("90% CI" = "blue")) +
  labs(title = "DS: Actual vs Forecasted", x = "", y = "DS (bps change)") +
  scale_x_date(date_breaks = "1 year", date_labels = "%Y") +
  theme_minimal() +
  theme(legend.title = element_blank(), legend.position = "bottom")
```



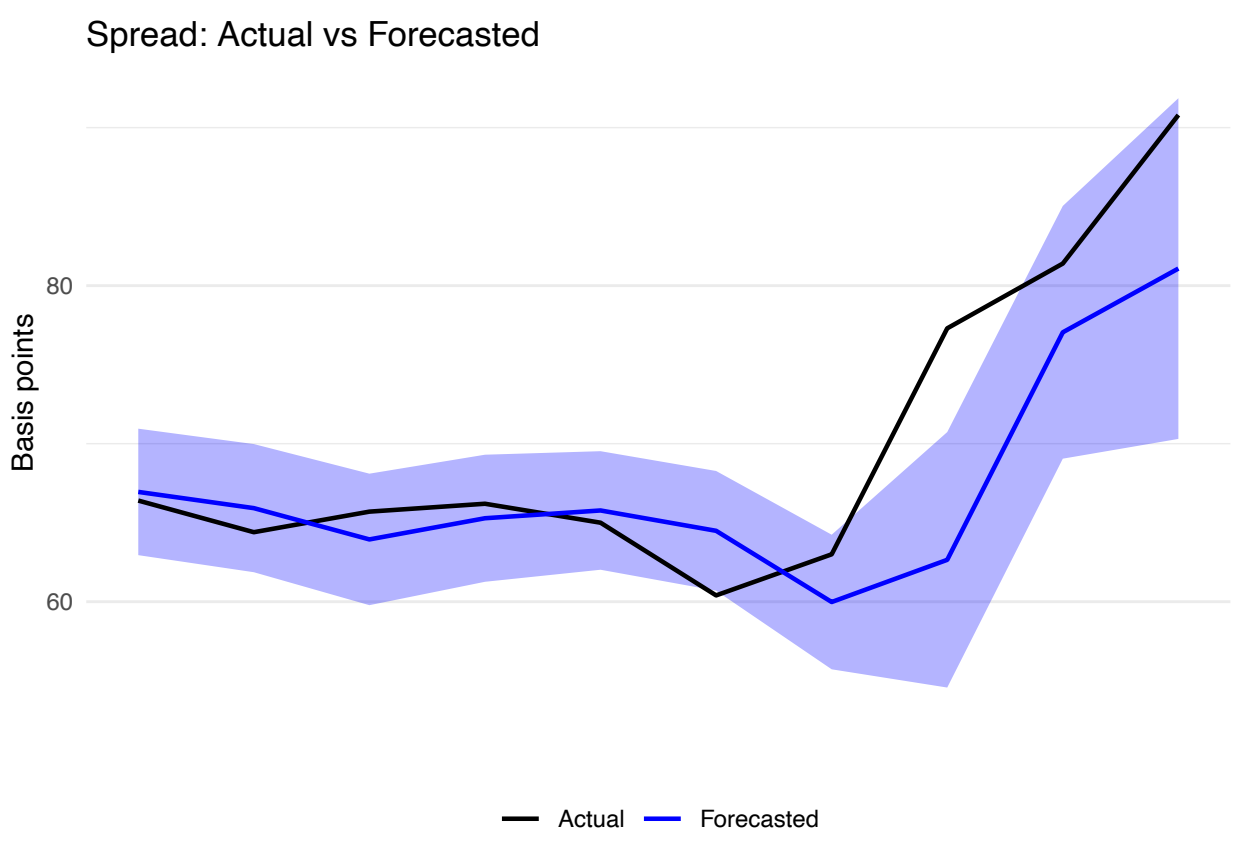
Back-conversion to Spread Level

```
get_spreads <- function() {
  fc_spread    <- numeric(n_test)
  lower_spread <- numeric(n_test)
  upper_spread <- numeric(n_test)

  spread_vec <- as.numeric(spread)

  for (i in 1:n_test) {
    prev_spread     <- spread_vec[n_train + i - 1]
    fc_spread[i]    <- prev_spread + forecast_df$pred_mu[i]
    lower_spread[i] <- prev_spread + forecast_df$ci_q05[i]
    upper_spread[i] <- prev_spread + forecast_df$ci_q95[i]
  }

  df <- data.frame(
    actual_spread = spread_vec[(n_train + 1):(n_train + n_test)],
    fc_spread     = fc_spread,
    ci_q05        = lower_spread,
    ci_q95        = upper_spread
  )
  rownames(df) <- rownames(forecast_df)
  return(df)
```

```r
}

df_spread <- get_spreads()
print(tail(df_spread))
```

```
##            actual_spread fc_spread   ci_q05   ci_q95
## 1971-06-13          65.0  65.77446 62.02278 69.52614
## 1971-06-14          60.4  64.48637 60.69917 68.27358
## 1971-06-15          63.0  59.98123 55.72218 64.24027
## 1971-06-16          77.3  62.65168 54.56720 70.73616
## 1971-06-17          81.4  77.04937 69.05258 85.04616
## 1971-06-18          90.8  81.08306 70.30304 91.86308
```

```r
df_spread$date <- as.Date(rownames(df_spread))

ggplot(df_spread, aes(x = date)) +
  geom_ribbon(aes(ymin = ci_q05, ymax = ci_q95), fill = "blue", alpha = 0.3) +
  geom_line(aes(y = actual_spread, color = "Actual"),    linewidth = 0.8) +
  geom_line(aes(y = fc_spread,     color = "Forecasted"), linewidth = 0.8) +
  scale_color_manual(values = c("Actual" = "black", "Forecasted" = "blue")) +
  labs(title = "Spread: Actual vs Forecasted", x = "", y = "Basis points") +
  scale_x_date(date_breaks = "1 year", date_labels = "%Y") +
  scale_y_continuous(breaks = scales::breaks_width(20)) +
  theme_minimal() +
  theme(legend.title = element_blank(), legend.position = "bottom")
```



Future Forecast

Point forecast for the next week using the last `n_test` observations.

```
data_fit  <- tail(as.numeric(diff_spread), n_test)
last_date <- tail(index(as.xts(diff_spread)), 1)

future_fit <- ugarchfit(spec = garch_spec, data = data_fit, solver = "hybrid")
```

```
## Warning in .egarchfit(spec = spec, data = data, out.sample = out.sample, :
## ugarchfit-->waring: using less than 100 data
##  points for estimation
```

```
future_fc  <- ugarchforecast(future_fit, n.ahead = 1)

mean_fc  <- as.numeric(fitted(future_fc))
sigma_fc <- as.numeric(sigma(future_fc))
dist_fc  <- future_fit@model$modeldesc$distribution
df_shape <- as.numeric(coef(future_fit)["shape"])
probs    <- c(0.05, 0.25, 0.75, 0.95)

quantiles_fc <- mean_fc + sigma_fc * qdist(dist_fc, p = probs, mu = 0, sigma = 1, shape = df_shape)

future_fc_df <- data.frame(ds = mean_fc, vol = sigma_fc,
                           q_05 = quantiles_fc[1], q_95 = quantiles_fc[4])

spread_conversion <- function(value, q_05, q_95, data) {
  last_val <- as.numeric(tail(data, 1))
  data.frame(spread = last_val + value,
             q_05   = last_val + q_05,
             q_95   = last_val + q_95)
}

future_spread <- spread_conversion(future_fc_df$ds, future_fc_df$q_05, future_fc_df$q_95, spread)

cat("DS on last date:", as.character(last_date), ":", tail(as.numeric(diff_spread), 1), "\n")
```

```
## DS on last date: 2026-03-23 : 2.4
```

```
cat("DS forecast for next week:", as.character(last_date + 7), ":\n")
```

```
## DS forecast for next week: 2026-03-30 :
```

```
print(future_fc_df)
```

```
##         ds      vol      q_05     q_95
## 1 2.164131 2.441392 -1.838361 6.166622
```

```
cat("Spread on last date:", as.character(last_date), ":", as.numeric(tail(spread, 1)), "\n")
```

```
## Spread on last date: 2026-03-23 : 93.2
```

```r
cat("Spread forecast for next week:", as.character(last_date + 7), ":\n")
```

```
## Spread forecast for next week: 2026-03-30 :
```

```r
print(future_spread)
```

```
##     spread     q_05     q_95
## 1 95.36413 91.36164 99.36662
```

Simulation Forecast (10,000 paths)

Simulating 10,000 paths accounts for the GARCH shock uncertainty, not just the ARMA mean.

```r
simulation <- ugarchsim(
  future_fit,
  n.sim       = 1,
  m.sim       = 10000,
  startMethod = "sample"
)

mean_sim <- mean(fitted(simulation))
vol_sim  <- mean(sigma(simulation))
q_90     <- quantile(fitted(simulation), probs = c(0.05, 0.95))

cat(sprintf("Mean DS:  %.4f\n", mean_sim))
```

```
## Mean DS:  2.1529
```

```r
cat(sprintf("Mean Vol: %.4f\n", vol_sim))
```

```
## Mean Vol: 2.4414
```

```r
cat(sprintf("Quantiles 5%% | 95%%: %.4f | %.4f\n", q_90[1], q_90[2]))
```

```
## Quantiles 5% | 95%: -1.8271 | 6.1361
```

```r
sim_spread <- spread_conversion(mean_sim, unname(q_90[1]), unname(q_90[2]), spread)
cat("Spread forecast (simulation) for next week:", as.character(last_date + 7), ":\n")
```

```
## Spread forecast (simulation) for next week: 2026-03-30 :
```

```r
print(sim_spread)
```

```
##     spread     q_05     q_95
## 1 95.35288 91.37286 99.33609
```